DistTrain: Addressing Model and Data Heterogeneity with Disaggregated Training for Multimodal Large Language Models

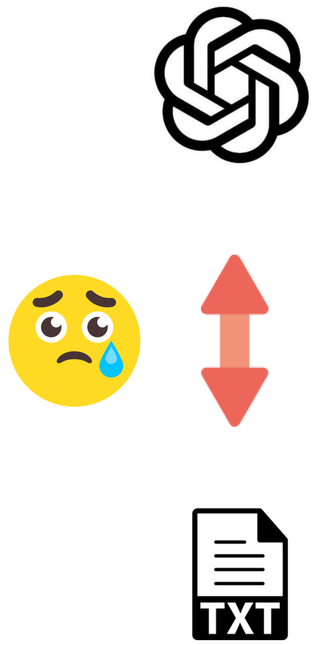
Zili Zhang*, Yinmin Zhong*, Yimin Jiang★, Hanpeng Hu† , Jianjian Sun†,

Zheng Ge†, **Yibo Zhu†**, Xin Jin*

* School of Computer Science, Peking University † **StepFun** ★ Independent Researcher

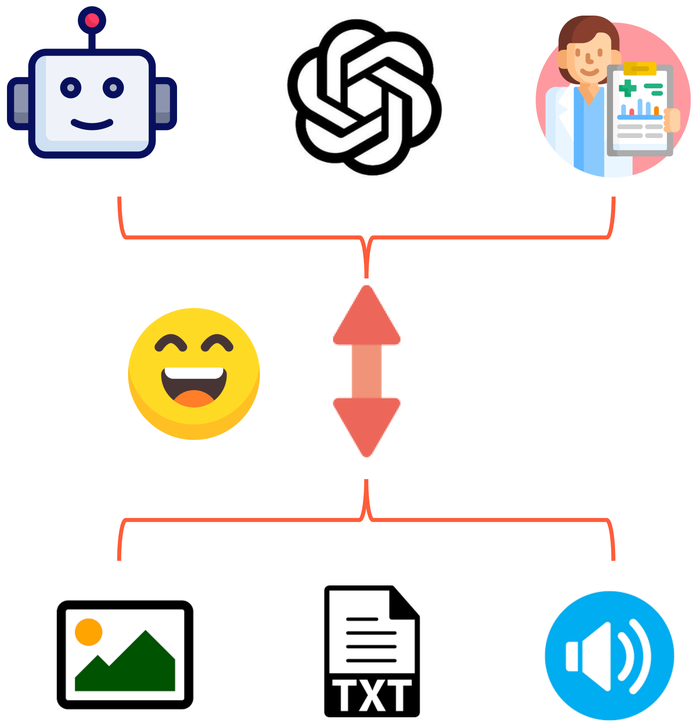
Multimodal LLM

Unimodal



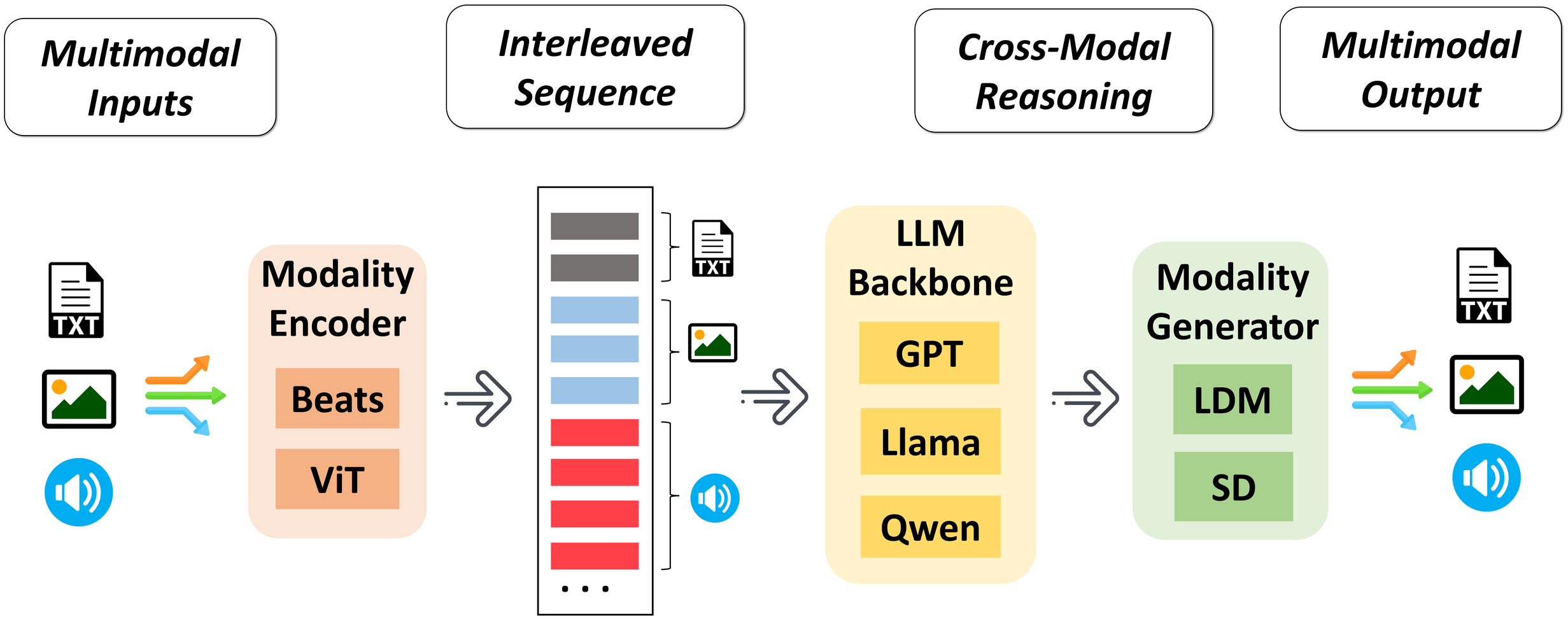
- *natural language*
- *code generation*

Multimodal



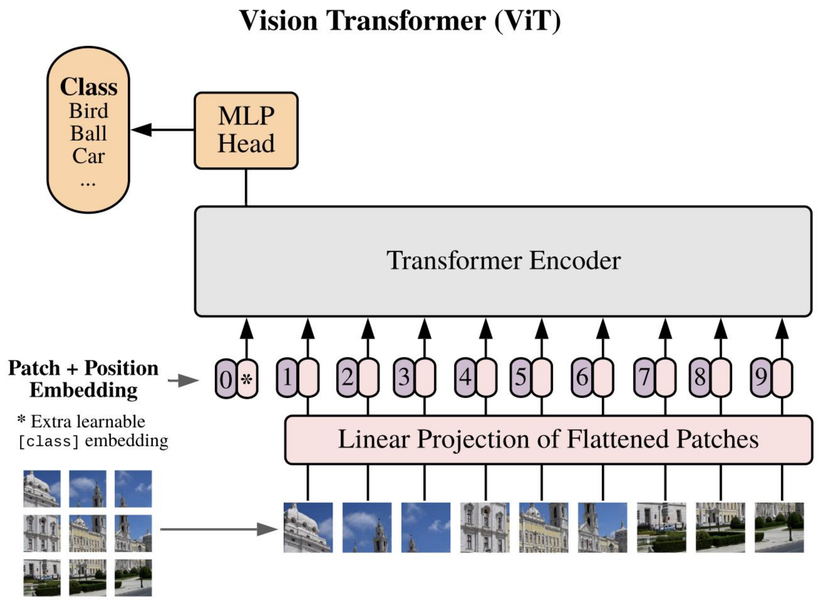
- *natural language, code*
- *photos, diagrams, charts*
- *speech, music, environmental sounds*

Multimodal LLM

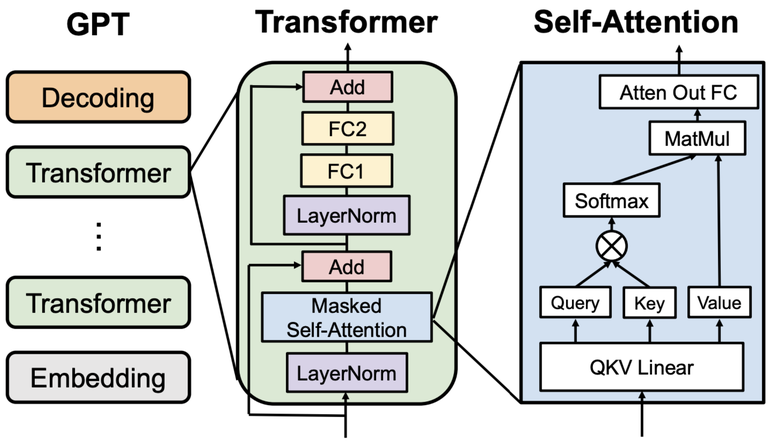


Model Heterogeneity

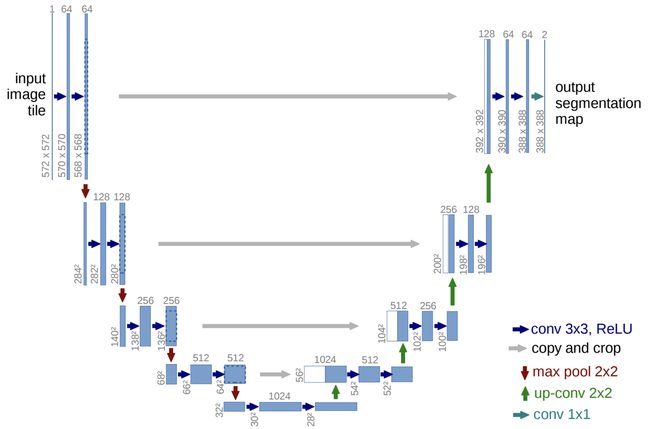
Modality Encoder



LLM Backbone



Modality Generator

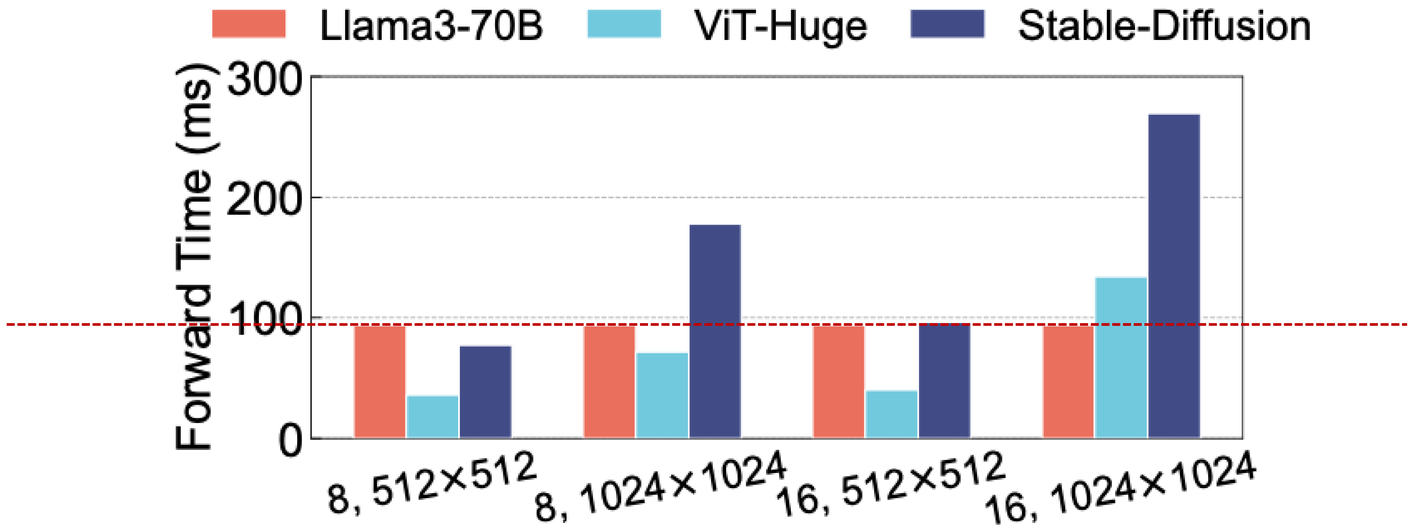


Lightweight Transformer

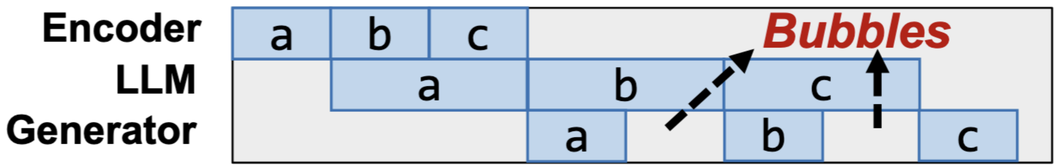
Heavyweight Transformer

Convolutional U-Net

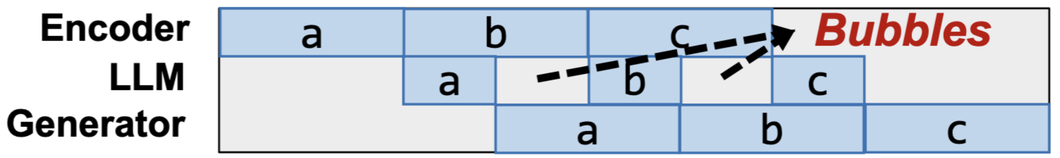
Model Heterogeneity



LLM Backbone is the PP straggler



Modality encoder/generator is the PP straggler

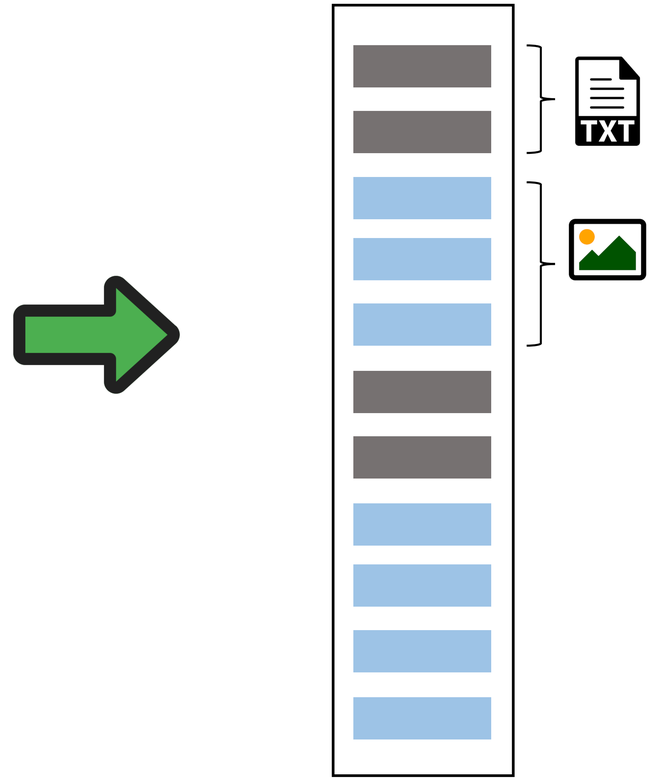


Data Heterogeneity

What is the landmark, and what plants surround it? Beside, remove the plants.

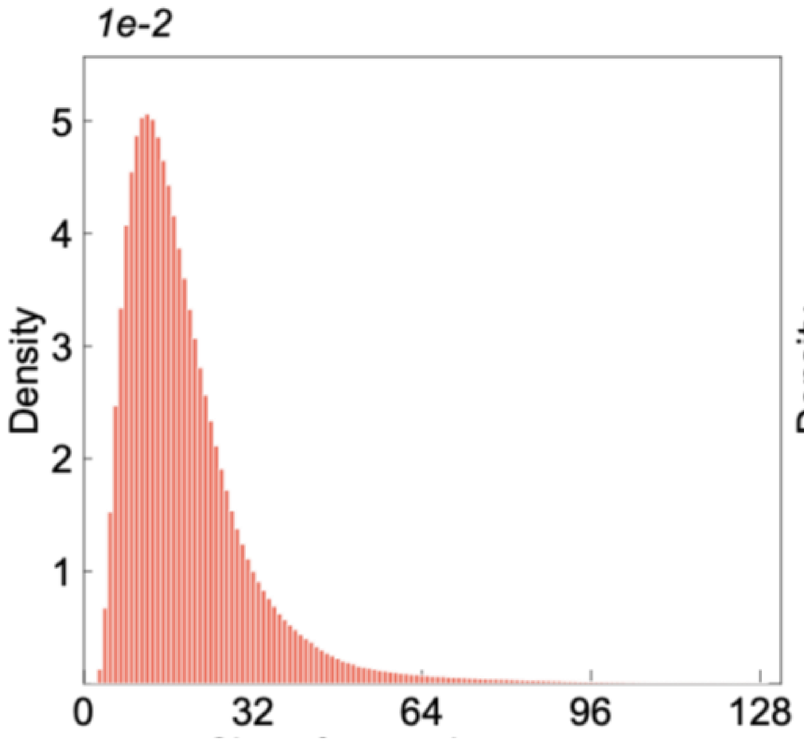
The landmark is the Great Wall, surrounded by flowers. Here is the edited image.

Training Sequence

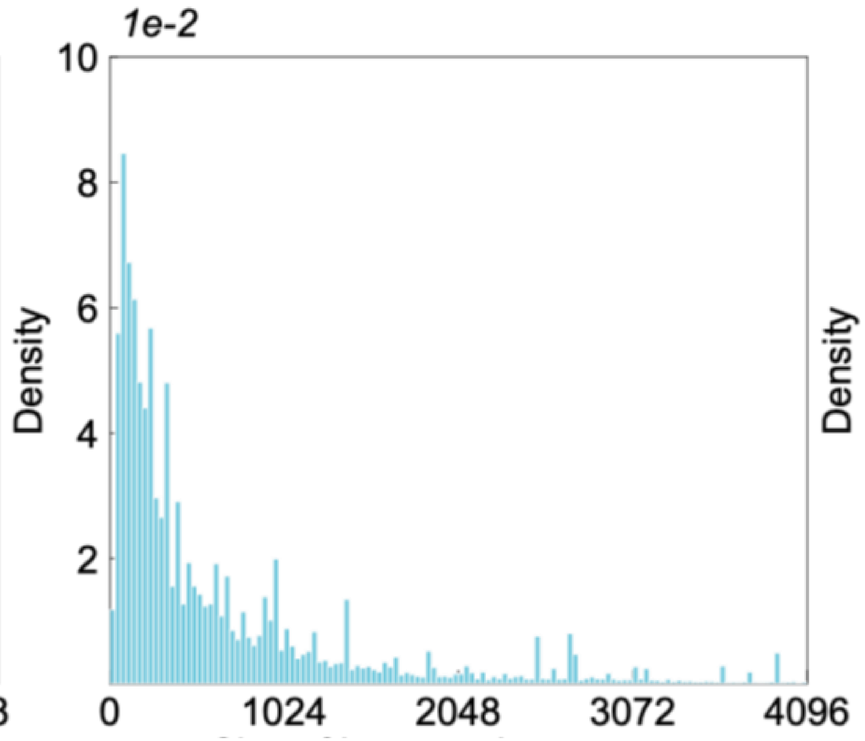


Data Heterogeneity

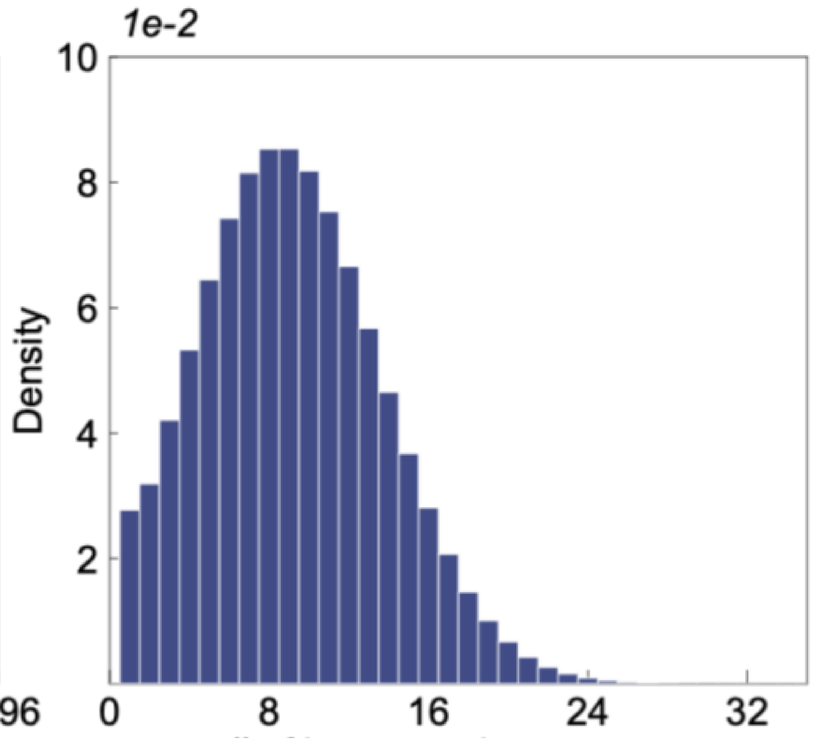
The sizes of text and image subsequences display highly skewed distributions



# of tokens for one text subsequence



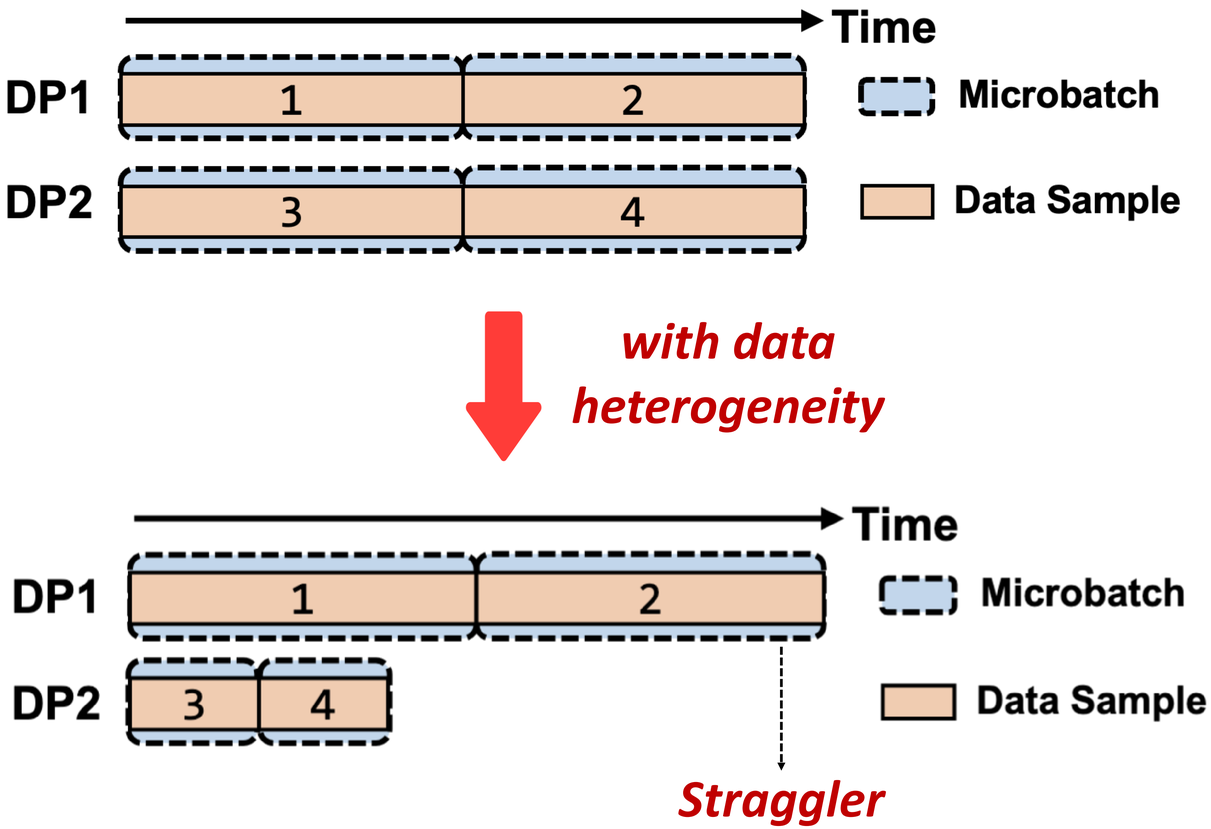
# of tokens for one image subsequence



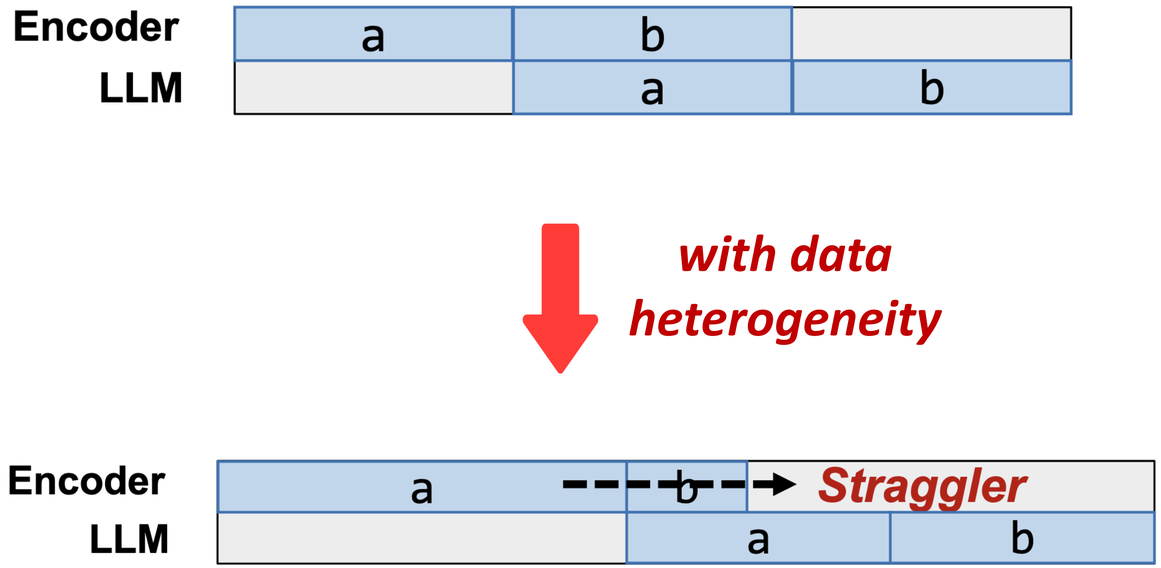
# of image subsequence in one 8K sequence

Data Heterogeneity

Intra-microbatch straggler



Inter-microbatch straggler



DistTrain design outline

Challenge 1: How to solve the model heterogeneity?

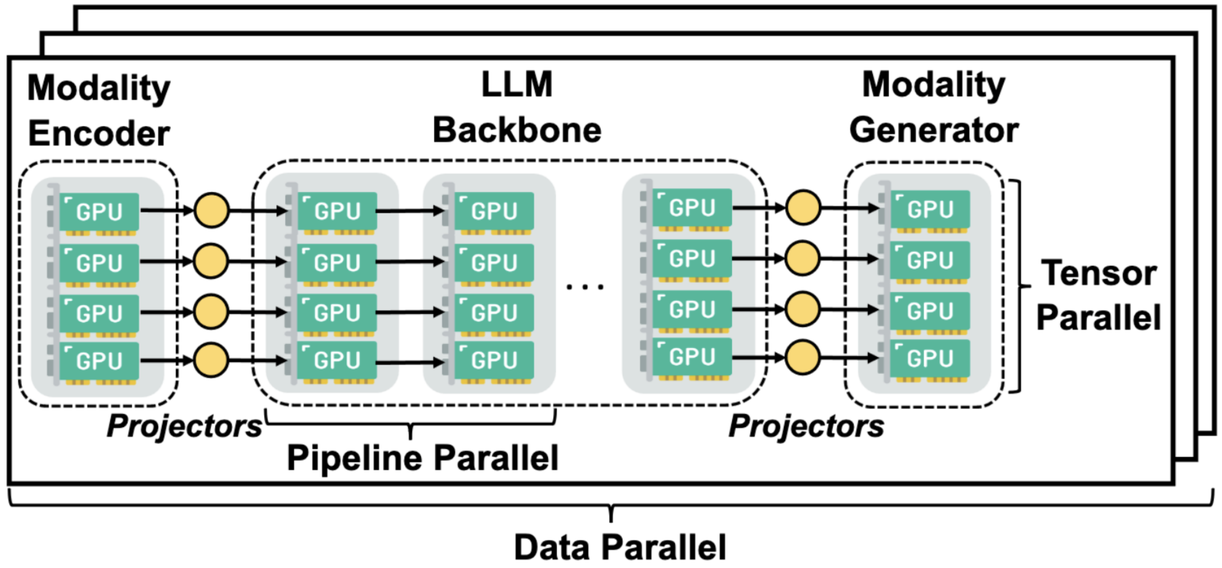
- Disaggregated model orchestration → Assign customized parallelism strategies to different modules
- Optimization formulation → Derive optimal parallelism configurations

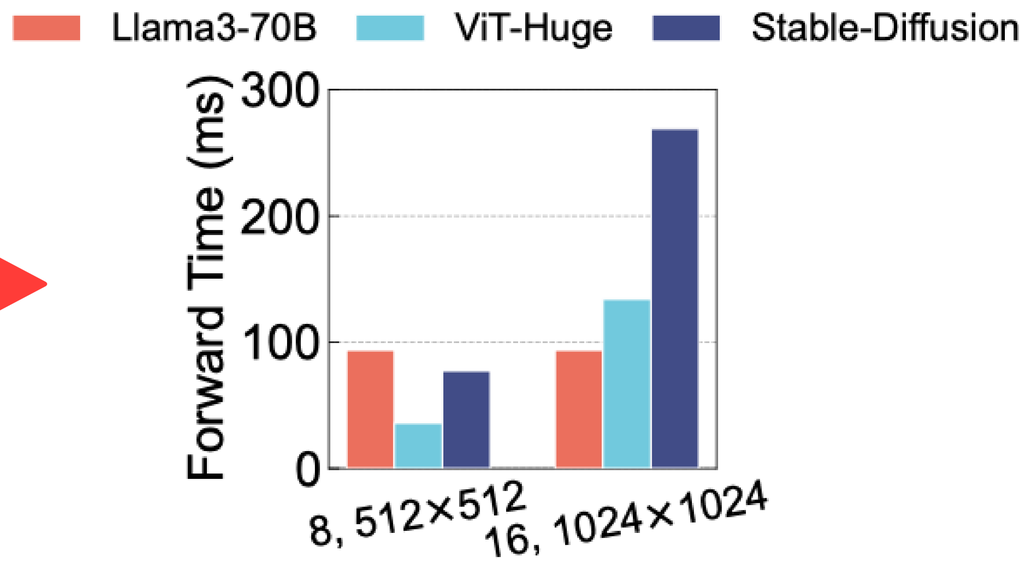
Challenge 2: How to solve the data heterogeneity?

- Intra-microbatch reordering → Mitigate stragglers across data-parallel instances
- Inter-microbatch reordering → Mitigate stragglers within pipeline parallelism

Addressing Model Heterogeneity

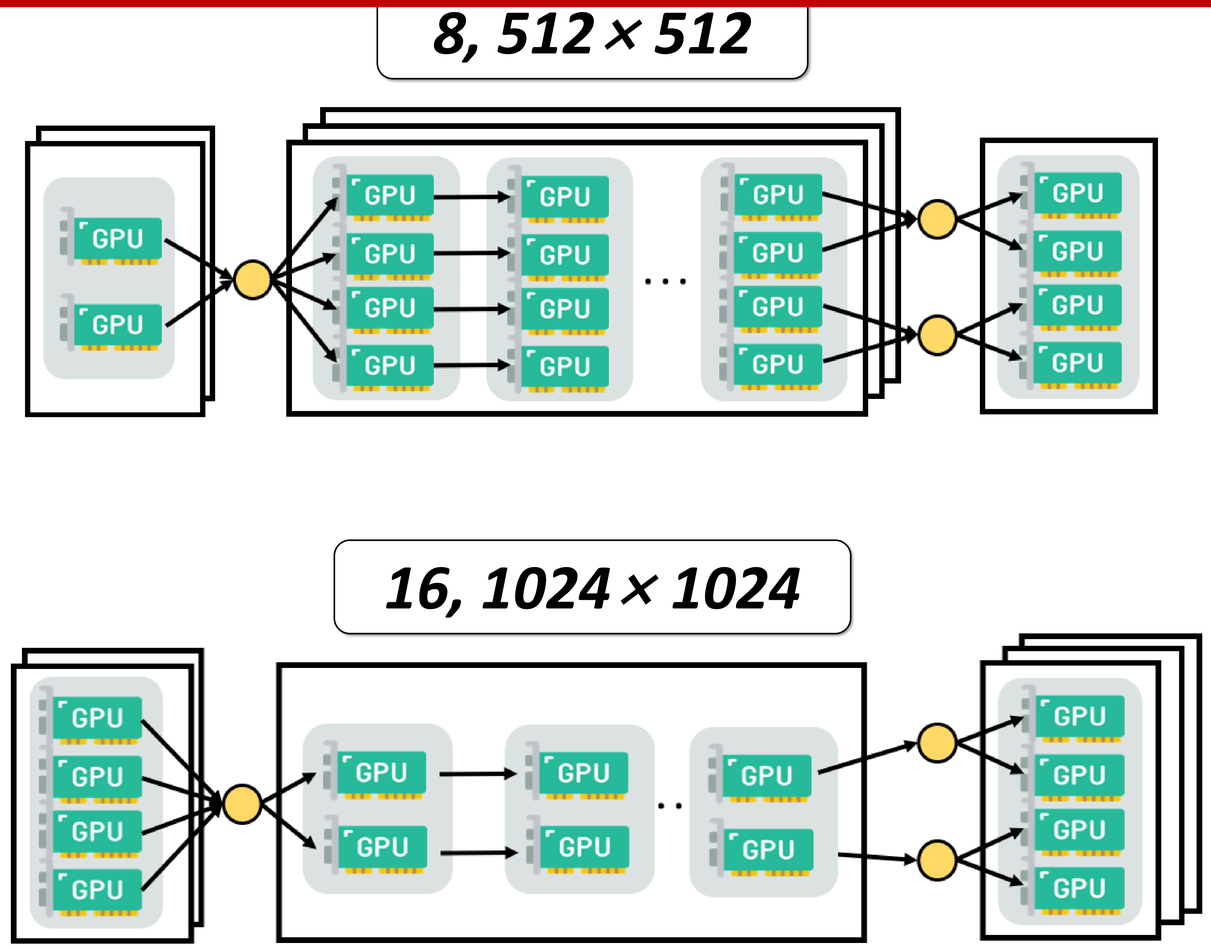
The monolithic orchestration incurs severe imbalance across modules 😢

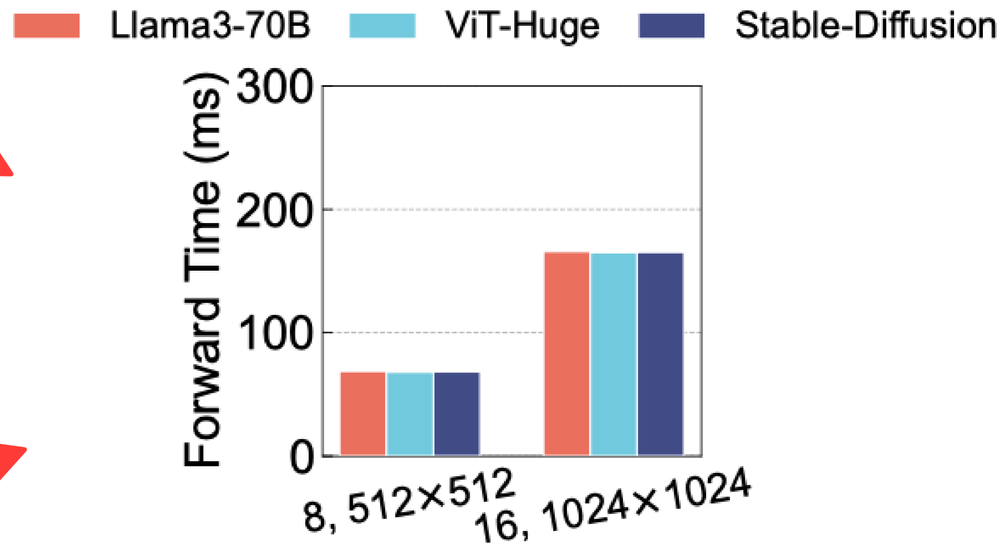




Addressing Model Heterogeneity

Our disaggregated orchestration balances computational loads 😄

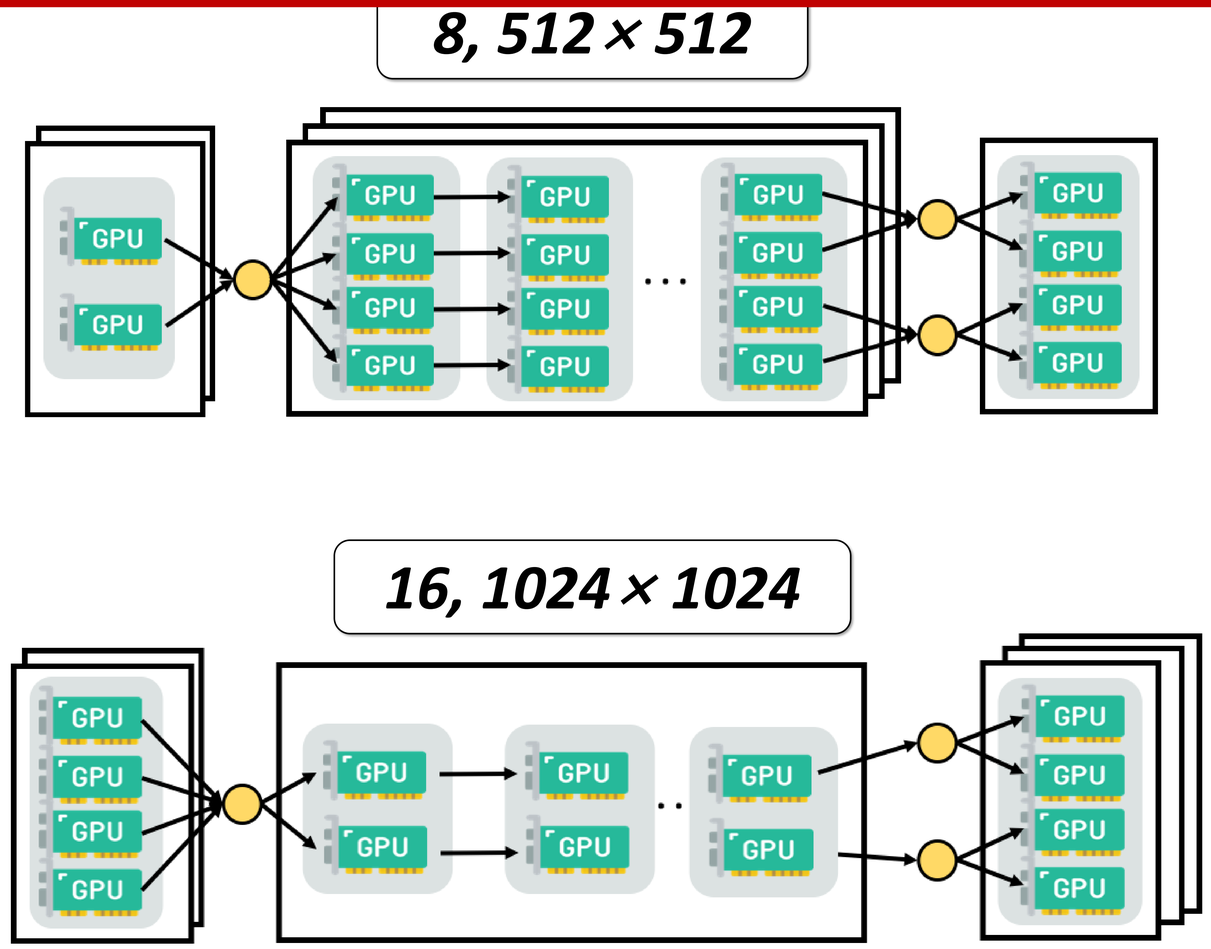


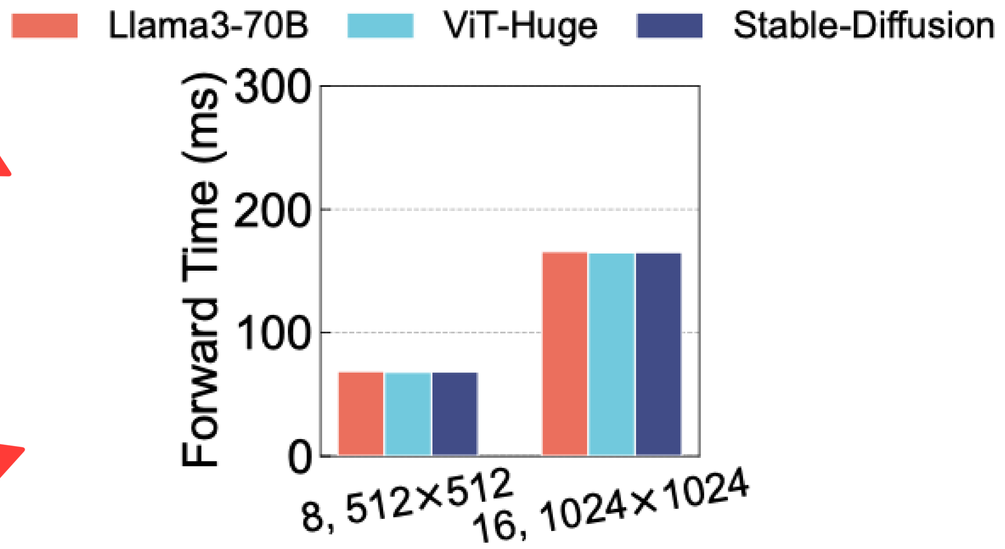


How to decide the detailed disaggregated parallelism strategy?

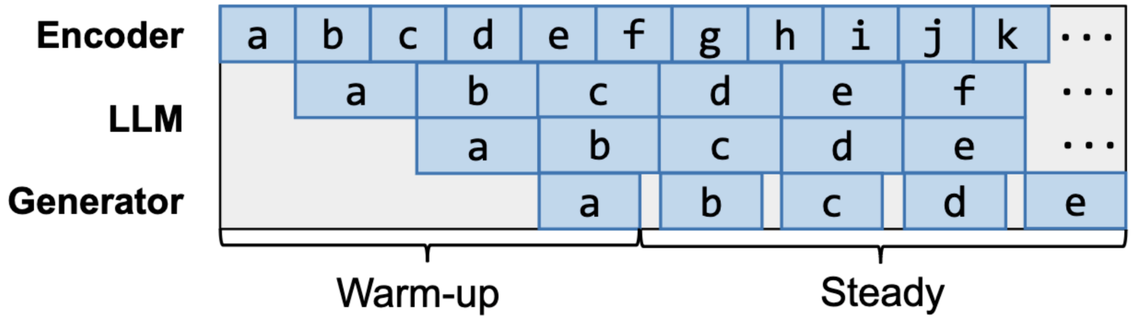
Addressing Model Heterogeneity

How to decide the detailed disaggregated parallelism strategy?





Addressing Model Heterogeneity



$$T_{warmup} = M \times C_{lm}(TP_{lm}) + \frac{DP_{lm} \times M}{DP_{me}} \times C_{me}(TP_{me}) + \frac{DP_{lm} \times M}{DP_{mg}} \times C_{mg}(TP_{mg})$$

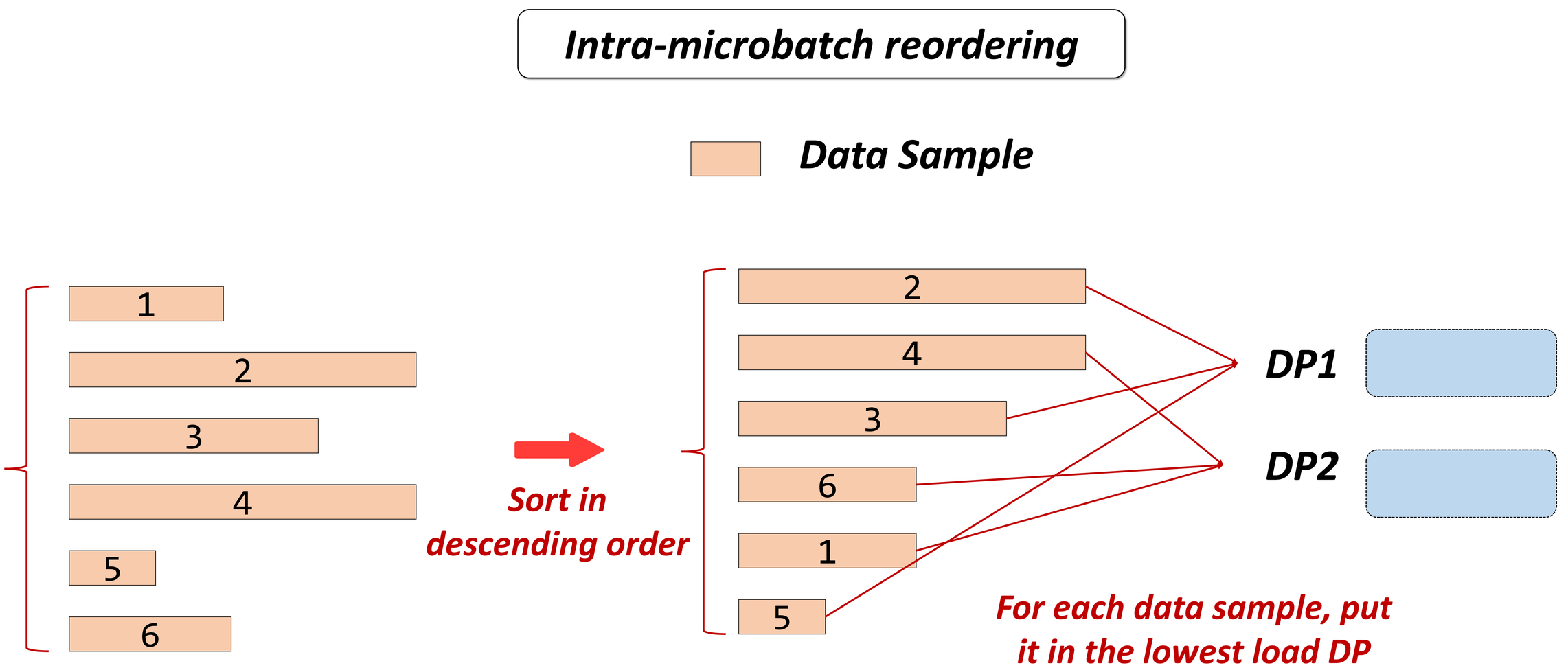
Time for the first microbatch to complete

$$T_{steady} = max \left\{ \begin{array}{l} \frac{DP_{lm} \times TP_{lm} \times M}{y} \times C_{lm}(TP_{lm}), \\ \frac{DP_{lm} \times TP_{me} \times M}{x} \times C_{me}(TP_{me}), \\ \frac{DP_{lm} \times TP_{mg} \times M}{z} \times C_{mg}(TP_{mg}) \end{array} \right\} \times \left(\frac{BS}{DP_{lm} \times M} - 1 \right)$$

Time of the slowest pipeline stage

# of microbatches

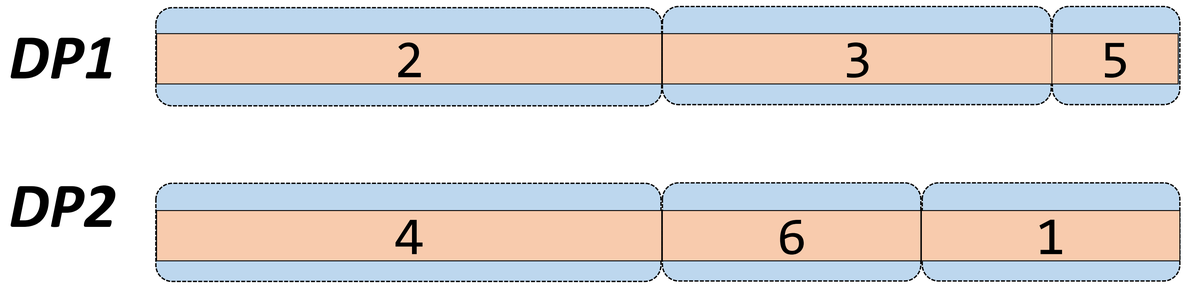
Addressing Data Heterogeneity



Addressing Data Heterogeneity

Intra-microbatch reordering

Data Sample

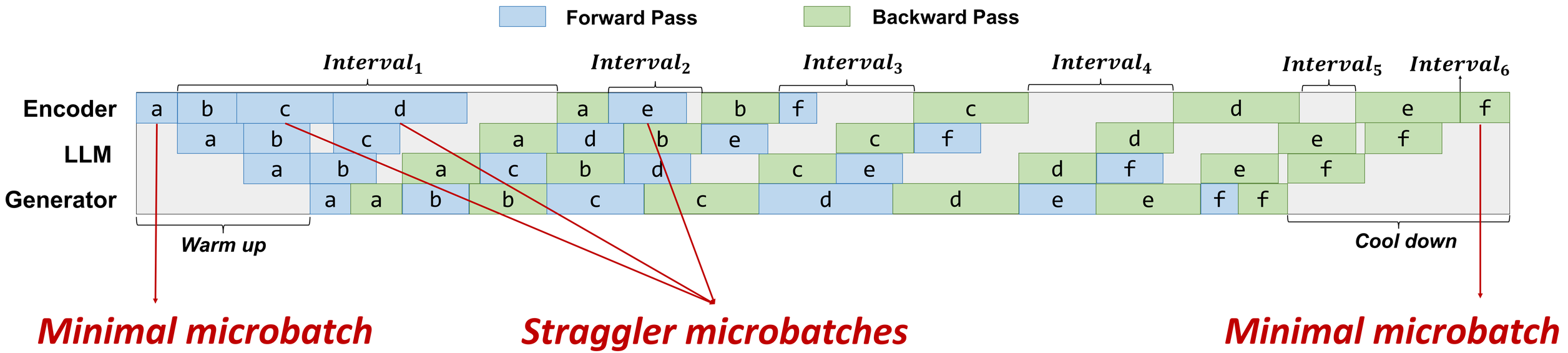


Balance !

Addressing Data Heterogeneity

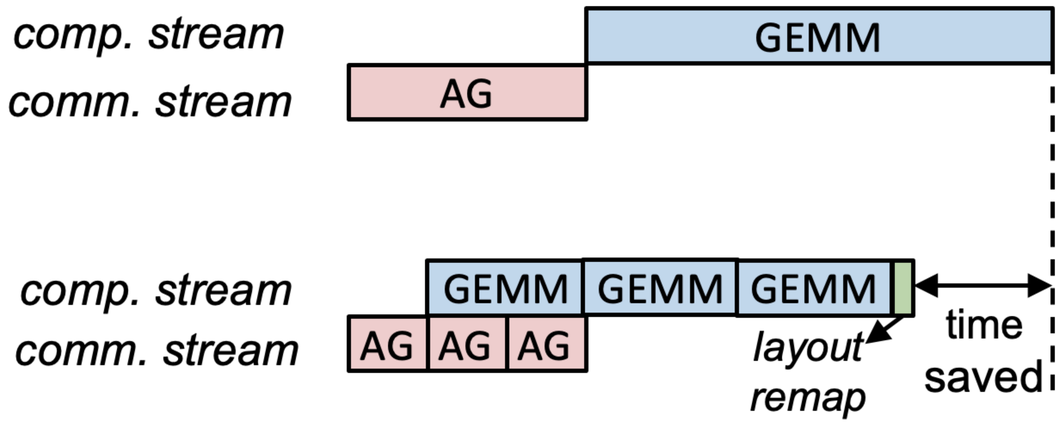
Inter-microbatch reordering

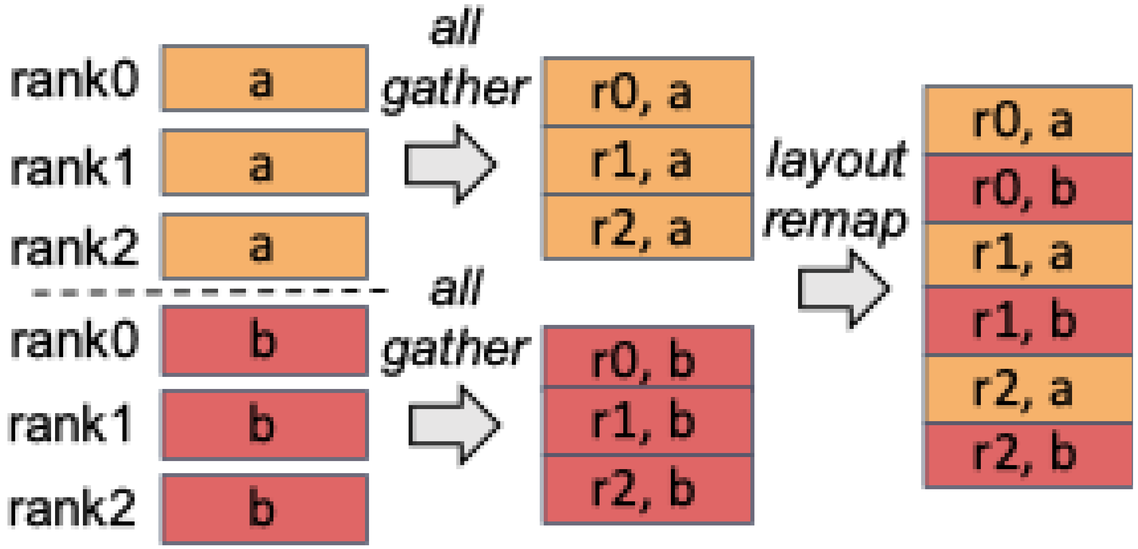
Hide the straggler within the large intervals in 1F1B pipeline



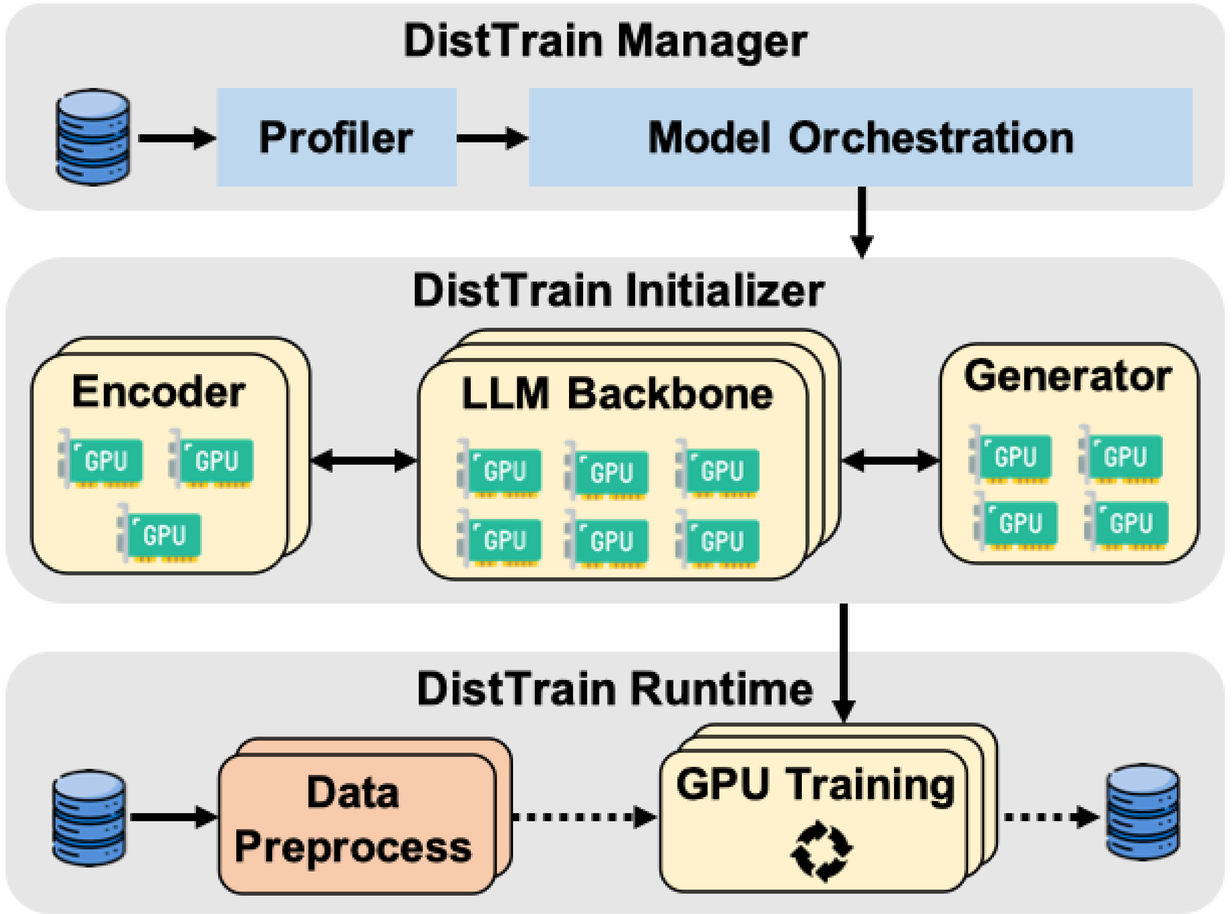
DistTrain System

StepCCL





DistTrain System



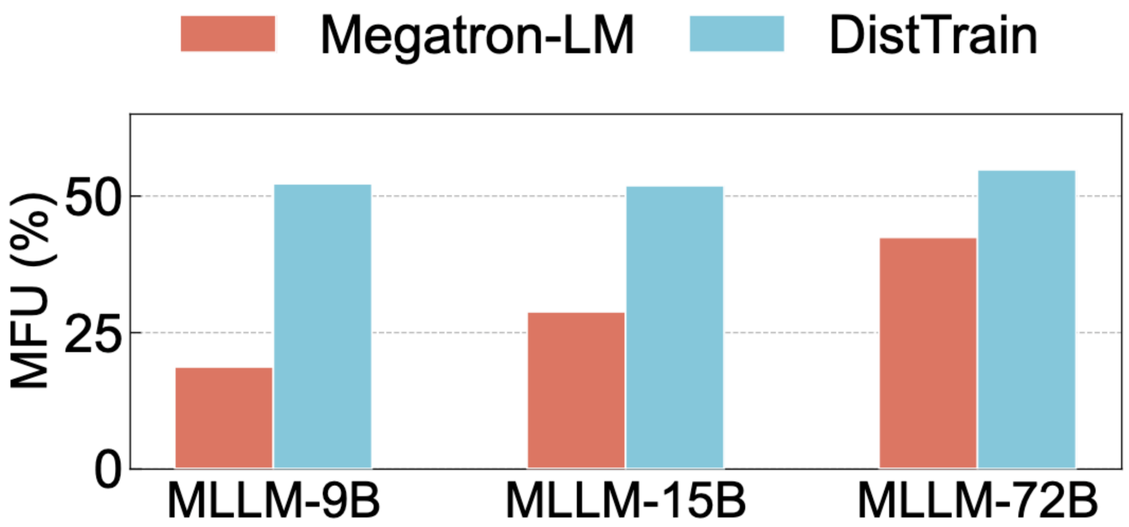
Evaluation

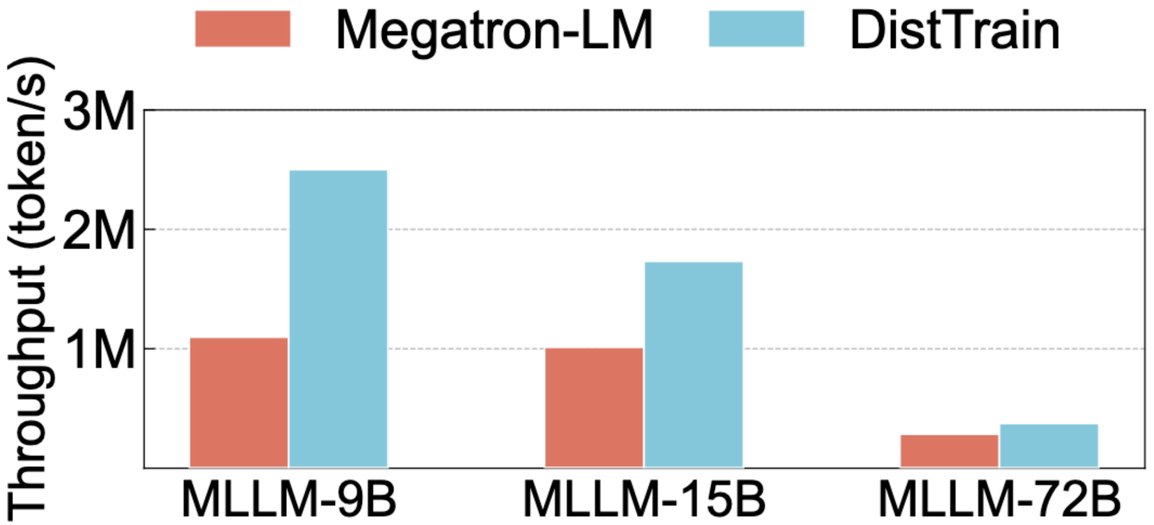
- Setup
  - Hardware: Production cluster with 1,296 NVIDIA Ampere GPUs
  - Network: NVLink and RDMA based on RoCEv
  - Dataset: Production training data from LAION
  - Model: MLLM (Llama + ViT + Stable-Diffusion)

Models	# of Layers	Hidden Size	FFN Hidden Size	# of Heads	# of Groups
Llama3-7B	32	4096	11008	32	32
Llama3-13B	40	5120	13824	40	40
Llama3-70B	80	8192	28672	64	8

Evaluation

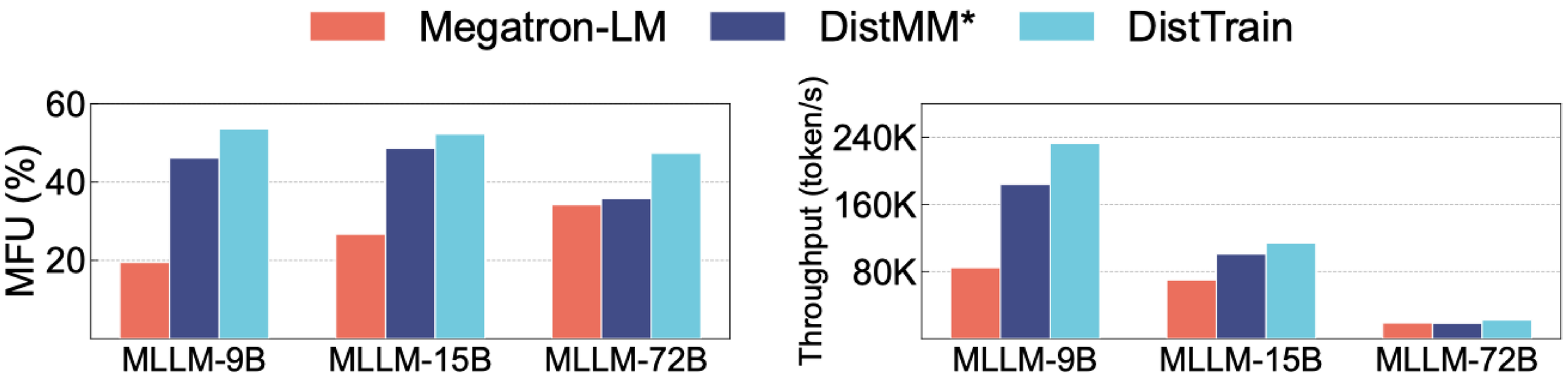
- DistTrain outperforms Megatron-LM on large-scale MLLM training





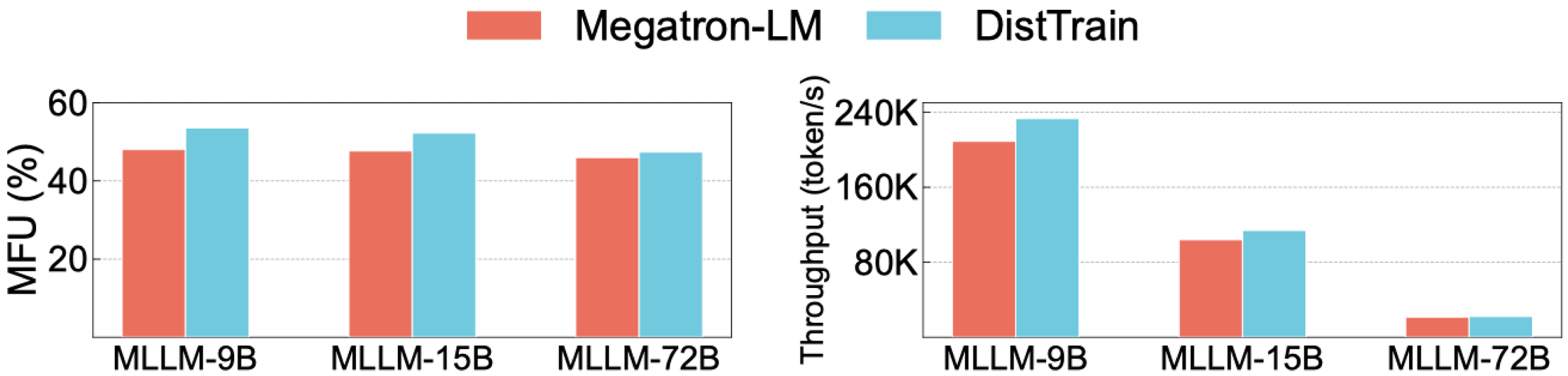
Evaluation

- DistTrain's disaggregated model orchestration is effective



Evaluation

- DistTrain's disaggregated data processing (reordering) is effective



Evaluation

- Performance under module freezing
- Performance of StepCCL
- System overhead of DistTrain

Conclusion

- Multimodal LLM training suffers from **model** and **data heterogeneity**

- DistTrain introduces two techniques:
  - *Disaggregated model orchestration* → tackles model heterogeneity
  - *Disaggregated data preprocessing* → tackles data heterogeneity

- DistTrain outperforms Megatron-LM by up to 2.2× on training throughput and achieves 54.7% MFU

- DistTrain supports large-scale **MLLM** (10B to 200B) training

Thank you!

zhuyibo@stepfun.com

MLLM Training Performance Optimization with DistTrain in Megatron-LM

Shifang Xu, DevTech, NVIDIA
Yu Huang, Solution Architect, NVIDIA

2026-03

Agenda

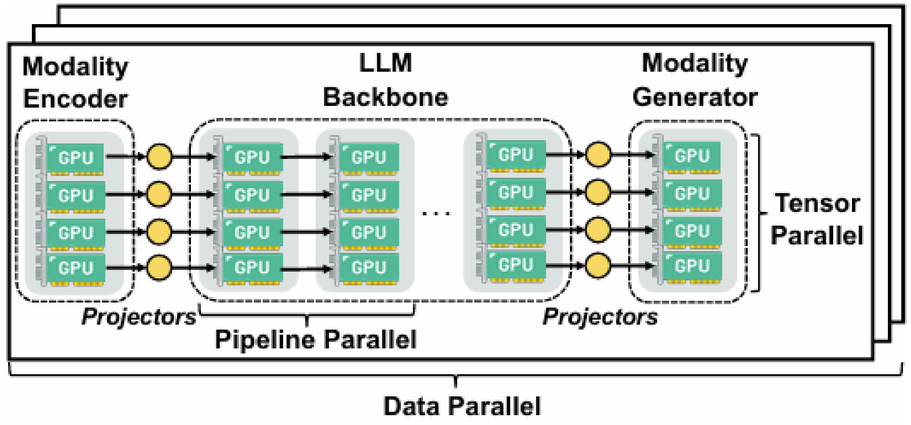
- How DistTrain is implemented
- How to use DistTrain
- Experimental results
- Future Works

Agenda

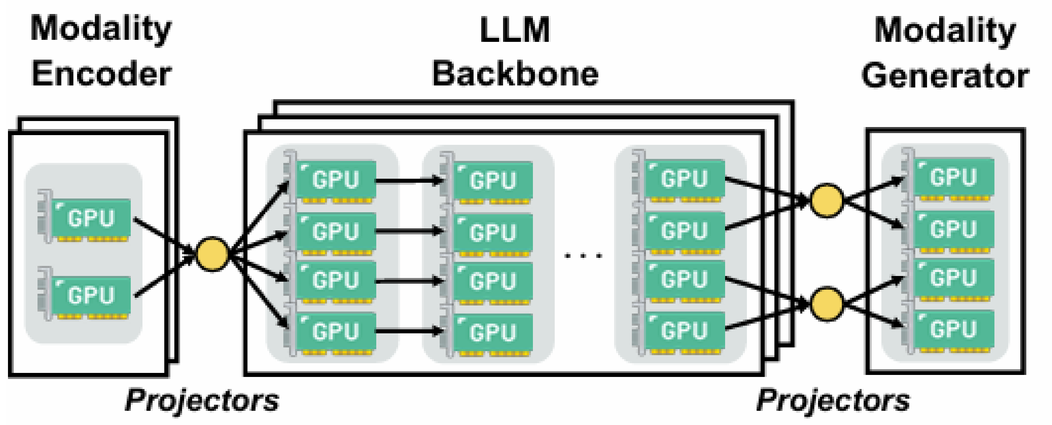
- How DistTrain is implemented
- How to use DistTrain
- Experimental results
- Future Works

DistTrain in Megatron-LM

Baseline



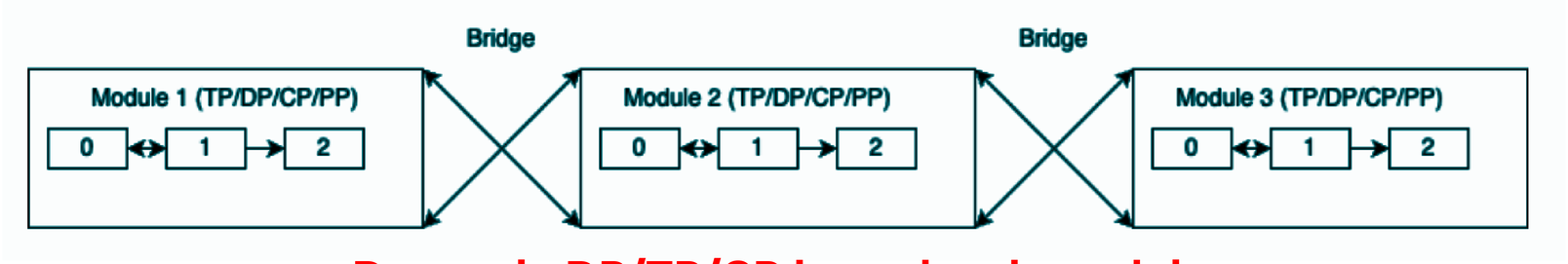
DistTrain



DistTrain Implementation in Megatron-LM

Goal:

- Enable multi-module models to utilize different parallelization strategies (TP/PP/DP).
- Establish and manage the necessary Pipeline Parallelism (PP) communications between modules.



Decouple DP/TP/CP in each sub module

Megatron-LM Refactor:

1. **Parameterize PG:** Support arbitrary number of models each has its own parallelism mapping. link

Multi-module Pipeline:

1. **Bridge Communicator:** Manages boundary communications between modules. link
2. **Multi-module Communicator:** Integrates both intra- and inter-pipeline communications. link
3. **Multi-module Scheduling:** Orchestrates the intra- and inter-pipeline PP computation and communications. link
4. **Multi-module Train Loop:** Create PG collections and use them to initialize sub models. link

PG(Process Group) in Megatron-LM “parallel_state.py”

- TP group
  - Communication among the shards of a layer’s split matrix.
- PP group
  - Communication of activations and gradients between pipeline stages.
- DP/DP×CP group
  - Weight sharing and gradient sync across full-model replicas.
- CP group
  - Attention exchange between different chunks of the same sequence.
- EP / Expert TP / Expert DP group
  - Expert count split, expert weight split, and expert weight replication domains in MoE.

- Embedding / Position Embedding Groups
  - Embedding layers exist only on specific pipeline stages
  - Dedicated groups for broadcast/sync among embedding-holding ranks
- Distributed Optimizer Groups
  - Split data parallel domain into multiple optimizer instances
  - Each instance manages a shard of parameters/gradients
- FP8 Amax Groups
  - Amax reduction across ranks for FP8 activation scaling

NVIDIA.

Megatron-LM Refactor Goal

Deprecate parallel_state.py for parallel management,

Replace with

Parameterize Process Group(PG)

(Local ProcessGroupCollection config at link)

Parameterize PG(Process Group)

Create PG in user code

```
import megatron.core import HyperCommGrid, ProcessGroupCollection
grid = HyperCommGrid(
    shape=[tp_size, cp_size, dp_size, pp_size],
    dim_names=["tp", "cp", "dp", "pp"], ...
)
tp_pg = grid.create_pg(["tp"]), ...
pg_collection = ProcessGroupCollection(tp=tp_pg, pp=pp_pg)
```

Gather PGs into ProcessGroupCollection.

Make PG argument to class and functions, e.g.:

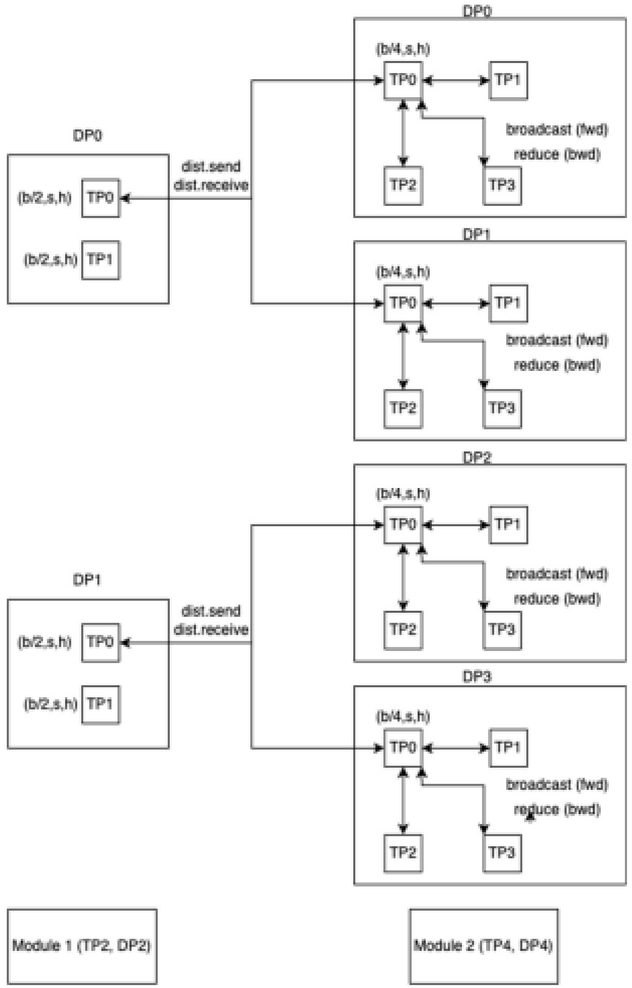
```
model = GPTModel(..., pg_collection=pg_collection)

class ColumnParallelLinear(torch.nn.Module):
  # Process group as an argument to the module
  def __init__(self, ..., tp_group):
    self.tp_group = tp_group
    #...

  def forward(...):
    # group needs to be passed to communication functions that need it
    gather_from_tensor_model_parallel_region(output_parallel, self.tp_group)
```

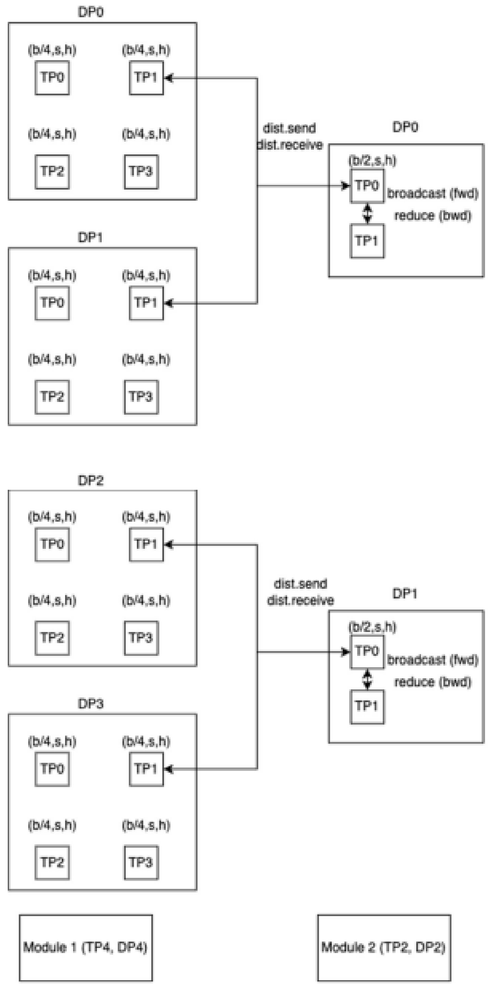
The model init function accepts ProcessGroupCollection as an argument.

Forward function accepts ProcessGroup as an argument.

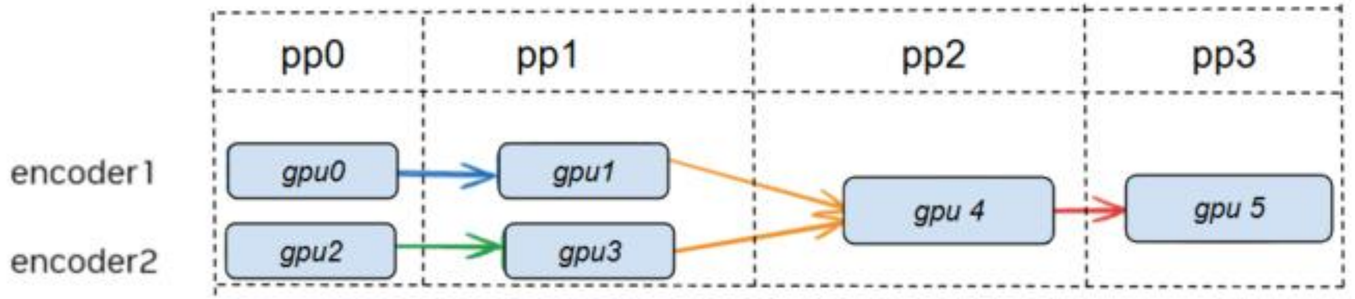
NVIDIA.

Bridge Communicator





Multi-module Communicator

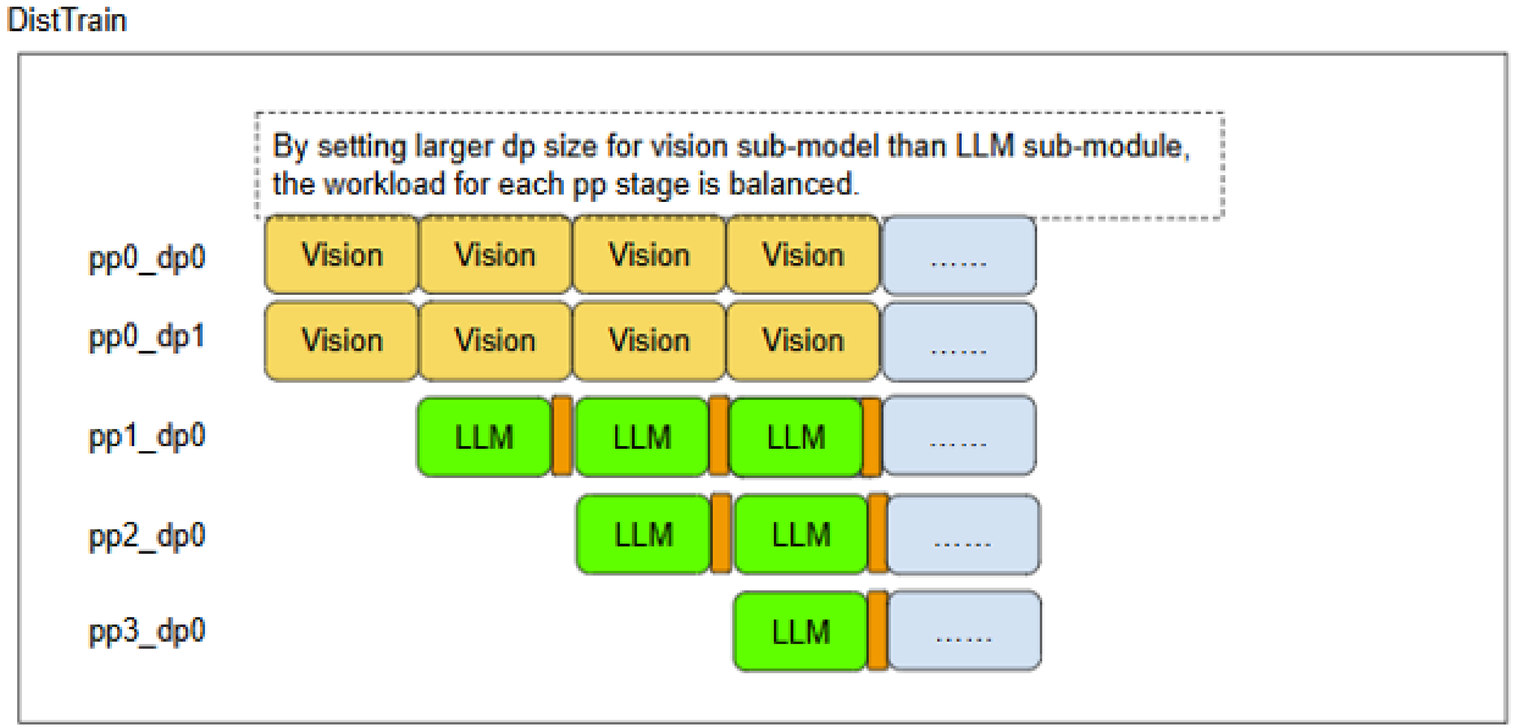


		gpu0	gpu2	gpu1	gpu3	gpu4	gpu5
warm up	recv_fwd	p2p	p2p	p2p	p2p	bridge	p2p
	send_fwd	p2p	p2p	bridge	bridge	p2p	p2p
warm up	send_fwd_recv_bwd	p2p	p2p	bridge	bridge	p2p	p2p
	send_bwd_recv_fwd	p2p	p2p	p2p	p2p	bridge	p2p
warm up	recv_bwd	p2p	p2p	bridge	bridge	p2p	p2p
	send_bwd	p2p	p2p	p2p	p2p	bridge	p2p

Multi-module Scheduling

For Vision:
PP size = 1
DP size = 2

For LLM:
PP size =3
DP size = 1



Agenda

- How DistTrain is implemented
- How to use DistTrain
- Experimental results
- Future Works

Source Code for Training QWen3-VL with DistTrain

- **Megatron-LM (https://github.com/NVIDIA/Megatron-LM/)**
  - **Bridge Communicator:** Manages boundary communications between modules.

https://github.com/NVIDIA/Megatron-LM/blob/main/megatron/core/pipeline_parallel/bridge_communicator.py

  - **Multi-module Communicator:** Integrates both intra- and inter-pipeline communications.

https://github.com/NVIDIA/Megatron-LM/blob/main/megatron/core/pipeline_parallel/multimodule_communicator.py

  - **Multi-module Scheduling:** Orchestrates the intra- and inter-pipeline PP computation and communications.

https://github.com/NVIDIA/Megatron-LM/pull/3129

- **Megatron-Bridge (https://github.com/NVIDIA-NeMo/Megatron-Bridge/)**
  - **QWen3-VL Model Define:** Defined the model architecture and the model forward function.

https://github.com/NVIDIA-NeMo/Megatron-Bridge/tree/main/src/megatron/bridge/models/qwen_vl

  - **Multi-module Train Loop:** Create PG collections and use them to initialize sub models.

https://github.com/NVIDIA-NeMo/Megatron-Bridge/pull/2367

DistTrain Arguments

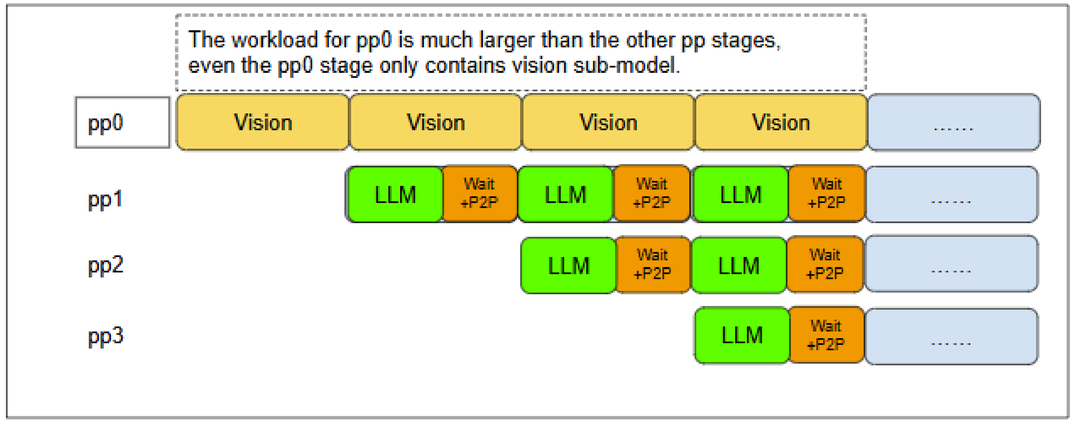
- **Argments related to DistTrain**
  - use_dist_train: bool = Falsed
  - vision_world_size: Optional[int] = None
  - language_world_size: Optional[int] = None
  - vision_tensor_model_parallel_size: Optional[int] = None
  - vision_pipeline_model_parallel_size: Optional[int] = None
  - vision_context_parallel_size: Optional[int] = None
  - vision_expert_tensor_parallel_size: Optional[int] = None
  - vision_expert_model_parallel_size: Optional[int] = None

- **For more information about DistTrain usage, please refer to the following link**

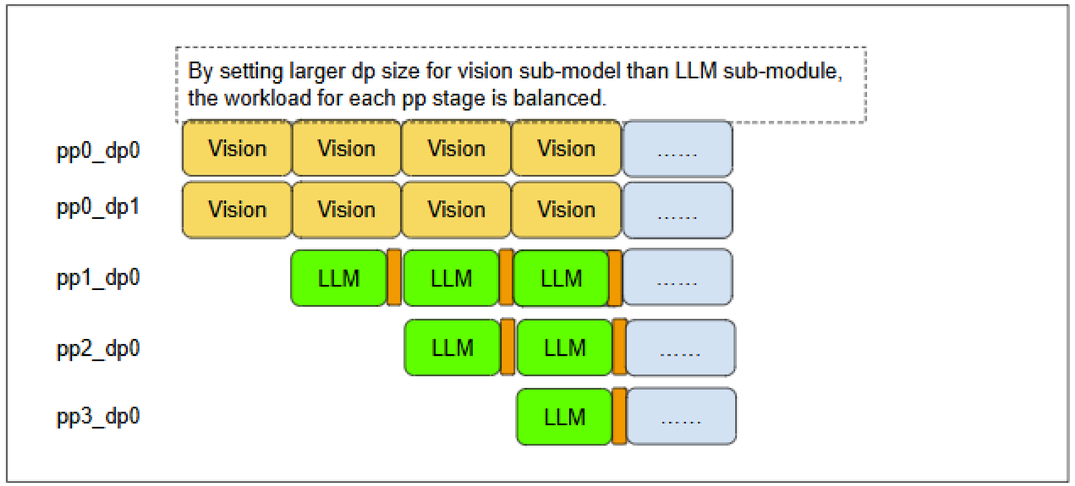
https://github.com/NVIDIA-NeMo/Megatron-Bridge/pull/2367

How To Choose Different Parallelism For Vision and LLM

Baseline

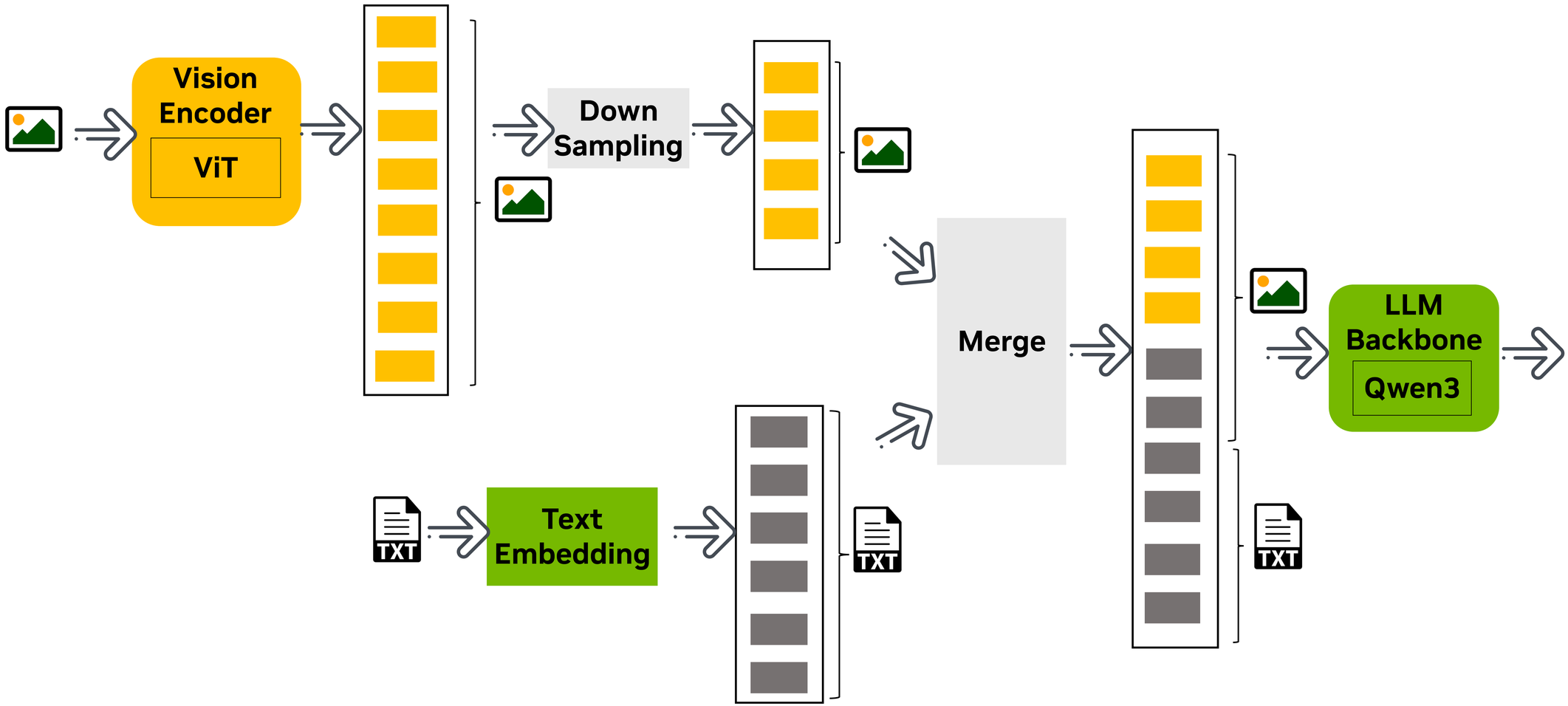


DistTrain

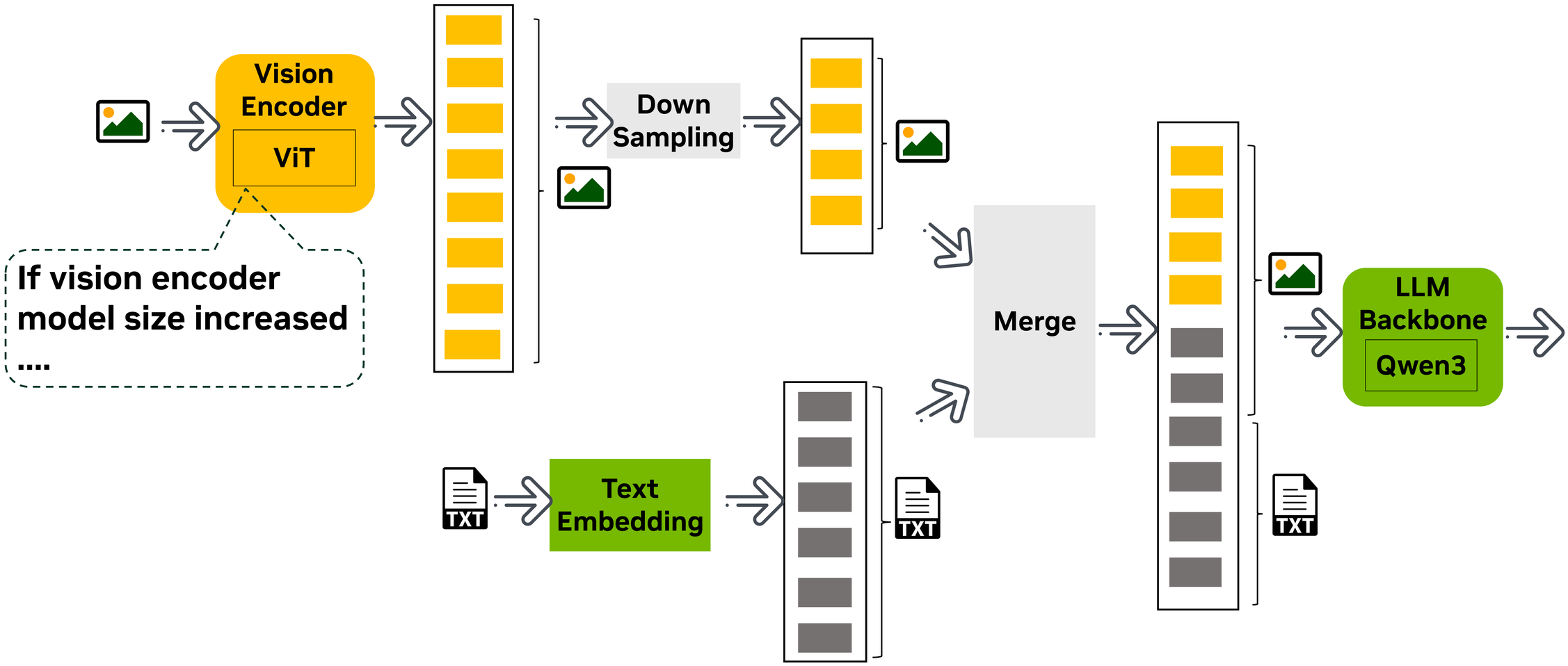


- DistTrain resolves Vision and LLM PP stages imbalance in MLLM training
  - Bubbles caused by the imbalance are major performance bottleneck.
  - DistTrain helps balance PP stages and improve efficiency.
- Extremely helpful for vision module scale up in MLLM training
  - ViT model size scale up
  - Vision token merge ratio and vision percentage increase in LLM input.
  - Vision token down sample ratio increase.

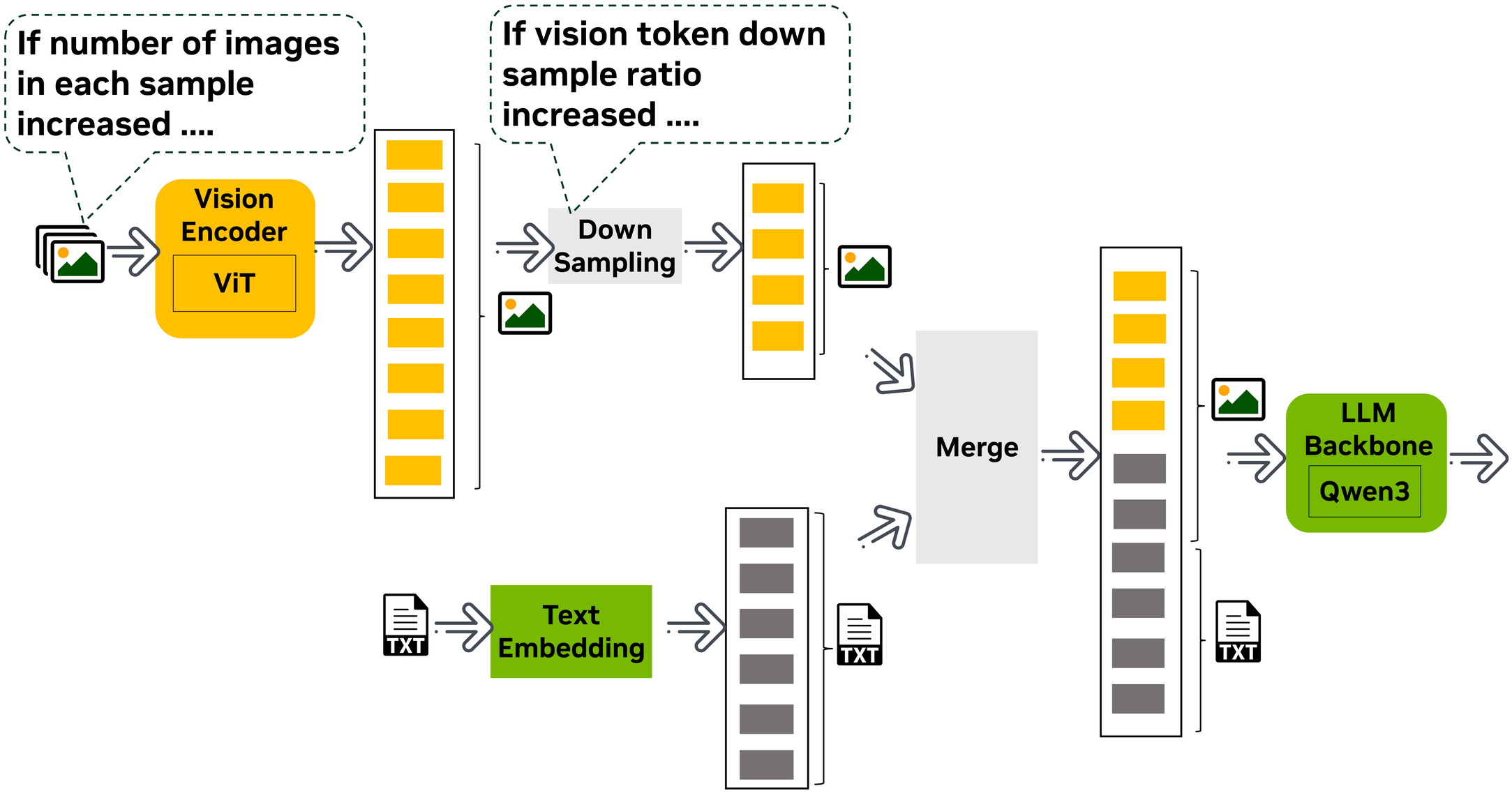
How To Choose Different Parallelism For Vision and LLM



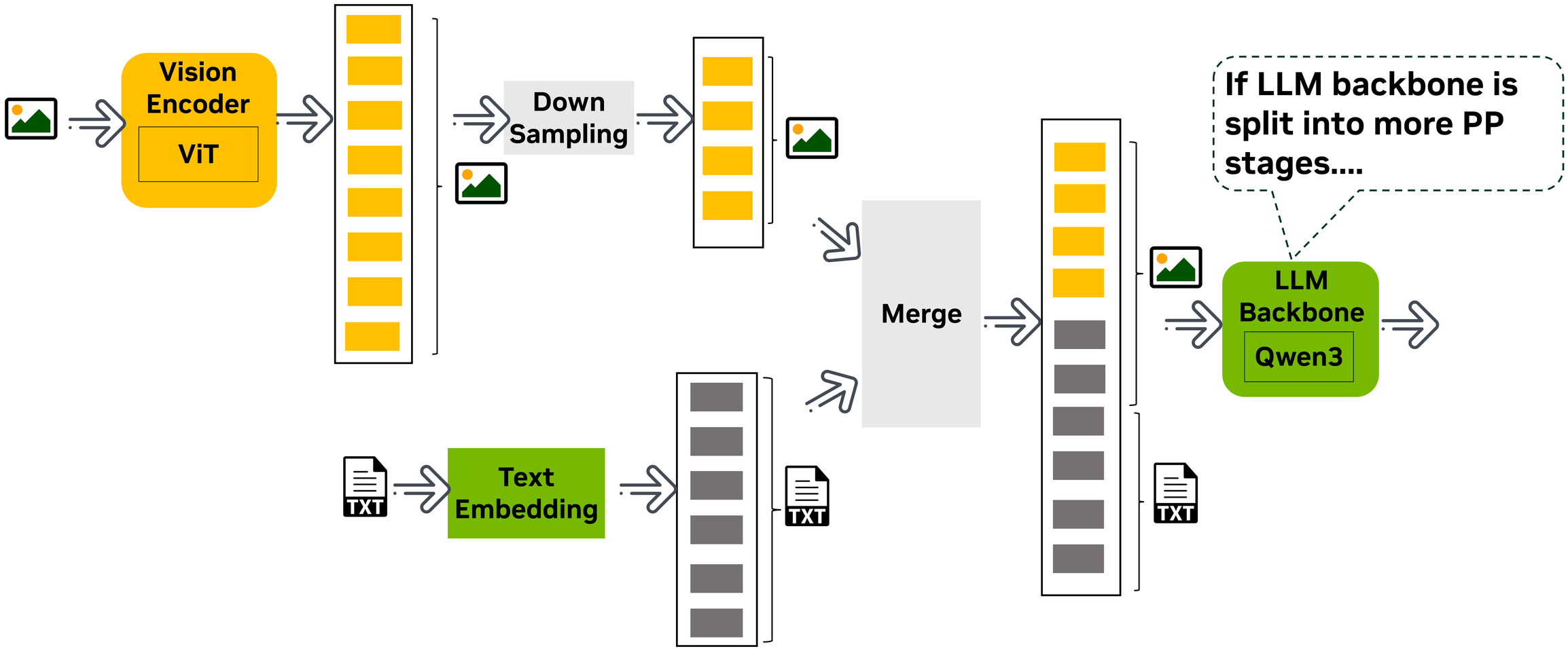
How To Choose Different Parallelism For Vision and LLM



How To Choose Different Parallelism For Vision and LLM



How To Choose Different Parallelism For Vision and LLM



How To Choose Different Parallelism For Vision and LLM

For QWen3-235B+ 2B ViT

LLM Model size	Vision Model size	# Images	down sample ratio	LLM SeqLen after merge and padding	# PP of LLM	Vision_Flops	LLM_Flops per_PP	Vision_Flops vs LLM_Flops per_PP
235B	0.6B	1	4	1024	4	1.27E+13	3.43E+13	0.37
		12	4	8192		1.52E+14	3.42E+14	0.45
		30	16	8192		3.81E+14	3.42E+14	1.11
		120	64	8192		1.52E+15	3.42E+14	4.46
		1	4	1024	8	1.27E+13	1.71E+13	0.74
		12	4	8192		1.52E+14	1.71E+14	0.89
		30	16	8192		3.81E+14	1.71E+14	2.23
		120	64	8192		1.52E+15	1.71E+14	8.92
	2B	1	4	1024	4	2.52E+13	3.43E+13	0.74
		12	4	8192		3.02E+14	3.42E+14	0.88
		30	16	8192		7.56E+14	3.42E+14	2.21
		120	64	8192		3.02E+15	3.42E+14	8.84
		1	4	1024	8	2.52E+13	1.71E+14	0.15
		12	4	8192		3.02E+14	1.71E+14	1.77
		30	16	8192		7.56E+14	1.71E+14	4.42
		120	64	8192		3.02E+15	1.71E+14	17.68

- Vision scale up includes: 1. larger vision module size, 2. scale up num images per sample and increase down sample rate.
- Vision scale up will lead to vision module FLOPs increase and may cause PP stage load imbalance.
- DistTrain will help in these green scenarios.
- Number of FLOPs is calculated according to Equation 3 in [2104.04473] Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM

Agenda

- How DistTrain is implemented
- How to use DistTrain
- Experimental results
- Future Works

DistTrain Testing with QWen3-235B+2B ViT

1.19x perf gain with 2x vision DP to LLM DP

Images per Micro-Batch	Image Merge Ratio	Vision Seq Len after Merge	LLM Seq Len after Padding	LLM PP	LLM DEP	Vision DP	Token/GPU/s	Perf Gain	Train Config
12	4	6912	8192	6	8	8	2041.62	1.0x	Baseline
12	4	6912	8192	6	8	16	2435.38	**1.19x**	DistTrain
12	4	6912	8192	6	8	24	2340.57	1.15x	DistTrain

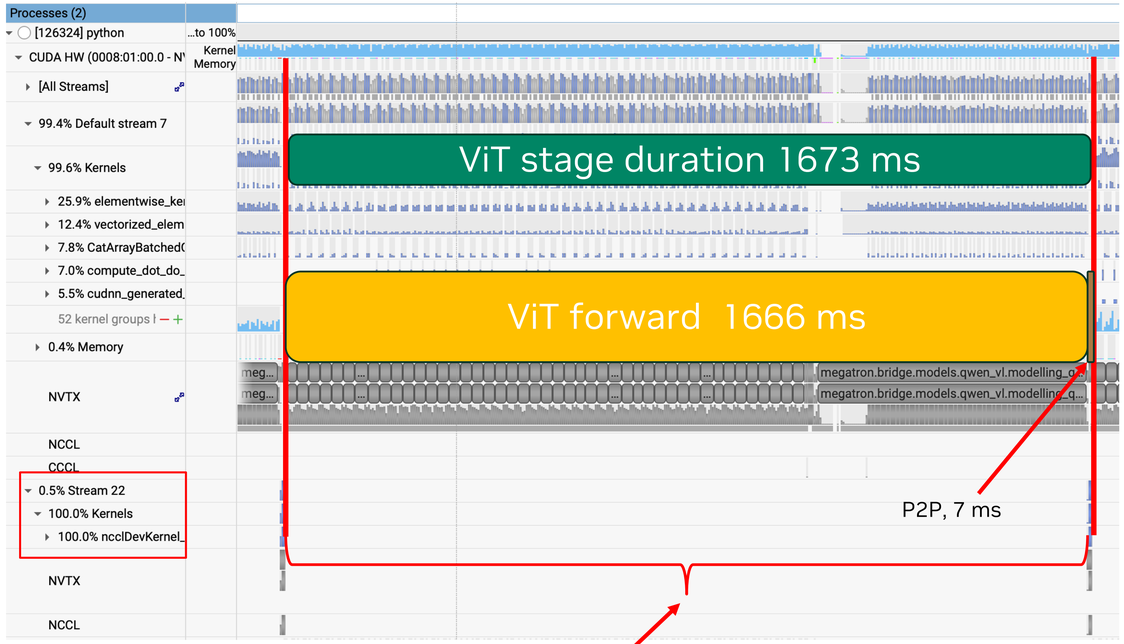
DistTrain Testing with Step3.5-196B+2B ViT

1.5x perf gain with 3x vision DP to LLM DP

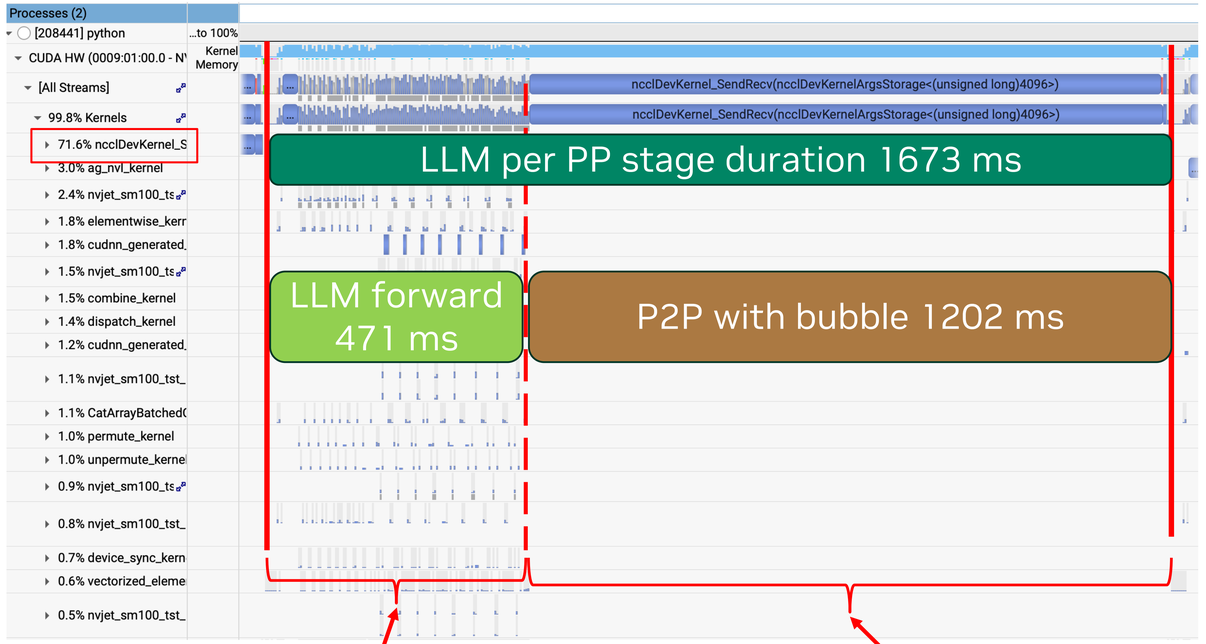
Images per Micro-Batch	Image Merge Ratio	Vision Seq Len after Merge	LLM Seq Len after Padding	LLM PP	LLM DEP	Vision DP	Token/GPU/s	Perf Gain	Train Config	
12	4	6912	8192	6	8	8	1461.13	1.0x	Baseline	
12	4	6912	8192	6	8	16	2106.02	1.44x	DistTrain	
12	4	6912	8192	6	8	24	2200.49	**1.51x**	DistTrain	
12	4	6912	8192	6	8	32	1987.88	1.36x	DistTrain	
12	4	6912	8192	6	8	48	1675.66	1.14x	DistTrain	

DistTrain Testing with Step3.5-196B+2B ViT

- Baseline with Vision DP8 + LLM PP6DEP8
  - 1673 ms per PP stage.
  - ViT module spend ~99% time in Compute.
  - LLM module spend ~71% time in P2P Send_Receive (**Bubbles! due to imbalanced PP stage**)



ViT Module, 99.5% time in transformer layers compute.

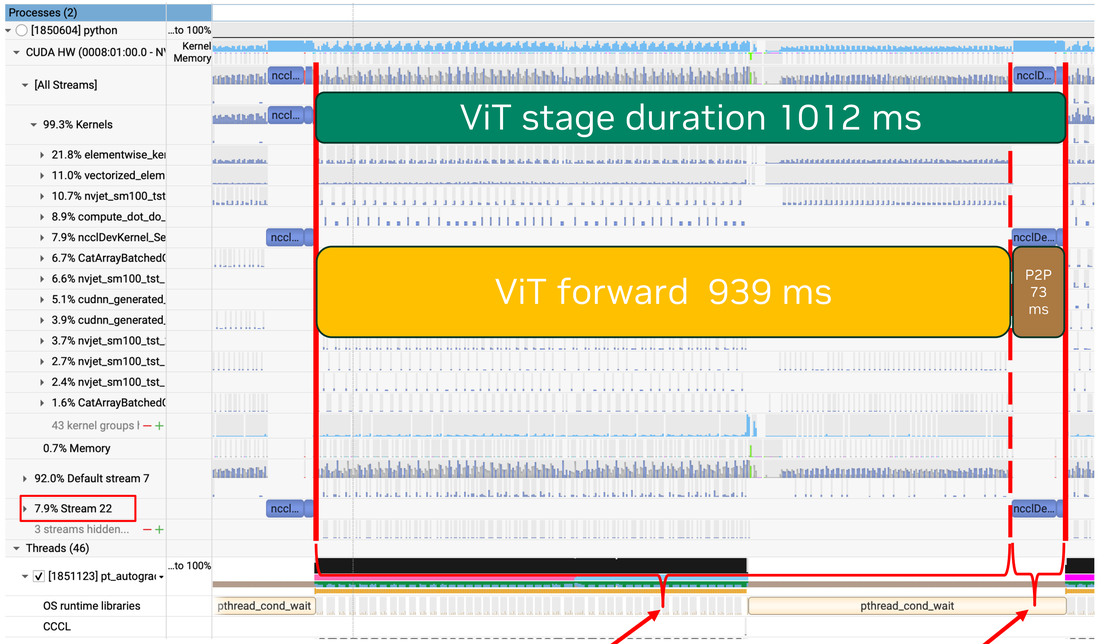


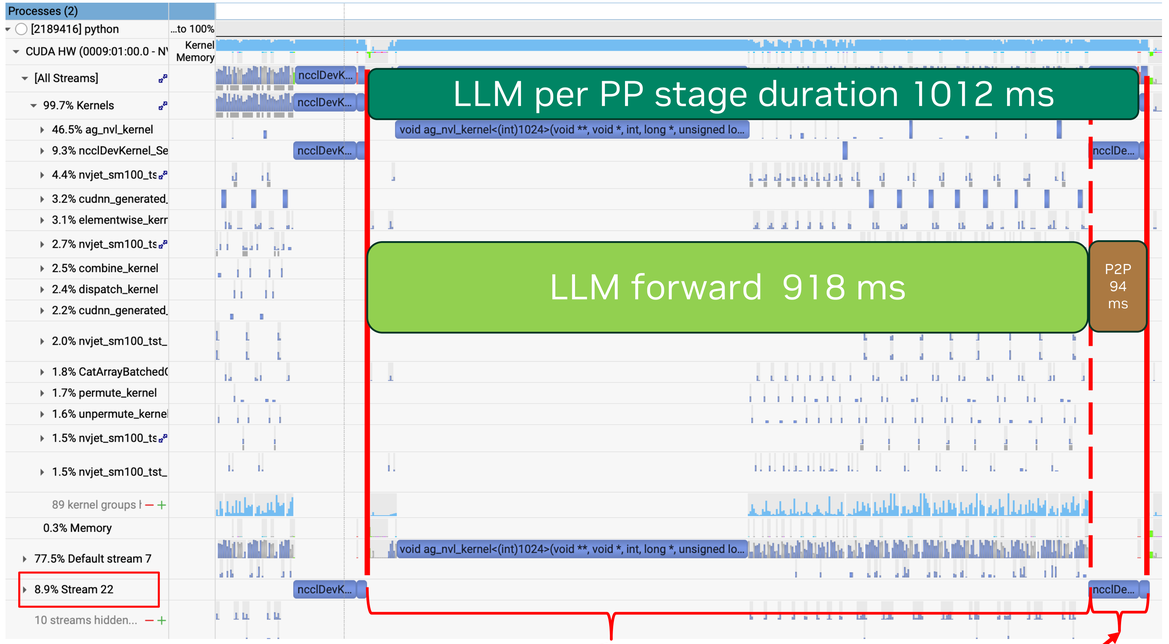
LLM Module, ~71% time in P2P bubbles.

LLM Module, ~29% time transformer layers compute.

DistTrain Testing with Step3.5-196B+2B ViT

- DistTrain with Vision DP24 + LLM PP6DEP8
  - 1012 ms per PP stage.
  - ViT compute expand to 24 DP ranks, ViT stage latency reduced, ~92% time in Compute.
  - LLM module ~91% time in Compute (**Bubbles reduced! Performance Improved with balanced PP stage**)





ViT Module, ~92% compute in transformer layers.

LLM Module, ~8% time in P2P bubbles.

LLM Module, ~91% time transformer layers compute.

LLM Module, ~9% time in P2P bubbles.

Agenda

- How DistTrain is implemented
- How to use DistTrain
- Experimental results
- Future Works

Future Works

- Decoupled Encoder and LLM stages
  - Similar to MDP which is used to training LongCat-Flash-Omni and Kimi 2.5
  - Benefit for post-training with freezed ViT.
- Heterogenous GPU utilized for different stages of MLLM training.
- Memory and Performance Estimator


NVIDIA
Thanks